

**TDK-thesis**

Bori Varga

# Drowsiness Detection Using Multimodal Neural Networks and Contrastive Learning

EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF ARTIFICIAL INTELLIGENCE



*Author:*

**Bori Varga**

Computer Science MSc

V. grade

*Supervisors:*

**Natabara Máté Gyöngyössy**

PhD Student

**Attila Bodnár**

Senior ML Engineer, PhD Student

Budapest, 2025

# Contents

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                                   | <b>5</b>  |
| <b>2</b> | <b>Literature review</b>                                                              | <b>7</b>  |
| 2.1      | Input Modalities for Drowsiness Detection . . . . .                                   | 8         |
| 2.2      | Representation Learning . . . . .                                                     | 9         |
| 2.2.1    | Motivation of Drowsiness Representation Learning . . . . .                            | 9         |
| 2.2.2    | Architectures for Representation Learning . . . . .                                   | 10        |
| 2.3      | Multimodal Learning . . . . .                                                         | 12        |
| 2.4      | Self-supervised Learning . . . . .                                                    | 13        |
| 2.4.1    | Masked Modeling . . . . .                                                             | 14        |
| 2.4.2    | EEG2Rep . . . . .                                                                     | 15        |
| 2.4.3    | Contrastive Learning . . . . .                                                        | 18        |
| 2.4.4    | CroSSL . . . . .                                                                      | 19        |
| 2.5      | Summary . . . . .                                                                     | 21        |
| <b>3</b> | <b>Methodology</b>                                                                    | <b>22</b> |
| 3.1      | Goals and Motivations . . . . .                                                       | 22        |
| 3.2      | DrowsiRank Architecture . . . . .                                                     | 24        |
| 3.3      | Model Components . . . . .                                                            | 25        |
| 3.3.1    | Unimodal Encoders . . . . .                                                           | 25        |
| 3.3.2    | Fusion Encoder . . . . .                                                              | 27        |
| 3.3.3    | Classifier Head . . . . .                                                             | 27        |
| 3.4      | Training Pipeline . . . . .                                                           | 28        |
| 3.4.1    | Patch-Level Reconstruction with Momentum Targets (PaReM) . .                          | 29        |
| 3.4.2    | Temporal Masked Modeling over Segment-Level Representations<br>(TMM-SegRep) . . . . . | 31        |
| 3.4.3    | Modality-Contrastive Alignment with Temporal Positives<br>(MoCA-T) . . . . .          | 34        |

|          |                                                                |           |
|----------|----------------------------------------------------------------|-----------|
| 3.4.4    | Classification . . . . .                                       | 38        |
| 3.5      | Data Preprocessing . . . . .                                   | 39        |
| 3.5.1    | Data Characteristics . . . . .                                 | 39        |
| 3.5.2    | Data Cleaning and Transformation . . . . .                     | 40        |
| 3.5.3    | Dataset Creation . . . . .                                     | 42        |
| <b>4</b> | <b>Implementation</b>                                          | <b>48</b> |
| 4.1      | Architecture Design . . . . .                                  | 48        |
| 4.1.1    | Unimodal Encoders . . . . .                                    | 48        |
| 4.1.2    | Temporal Masked Modeling . . . . .                             | 50        |
| 4.1.3    | Multimodal Fusion . . . . .                                    | 51        |
| 4.1.4    | Classification . . . . .                                       | 52        |
| 4.2      | Dataset Implementation . . . . .                               | 53        |
| 4.2.1    | Custom Dataset Structures . . . . .                            | 53        |
| 4.2.2    | Storage-Efficient Data Handling and Loading Strategy . . . . . | 54        |
| 4.3      | Training infrastructure . . . . .                              | 54        |
| 4.3.1    | Hardware and Training Environment . . . . .                    | 54        |
| 4.3.2    | Gradient Accumulation and Batch Management . . . . .           | 55        |
| 4.3.3    | Precision Handling . . . . .                                   | 56        |
| 4.3.4    | Training Loop and Logging Infrastructure . . . . .             | 56        |
| 4.3.5    | Reproducibility . . . . .                                      | 57        |
| 4.3.6    | Checkpointing and Weight Loading . . . . .                     | 57        |
| 4.3.7    | Optimization and Parameter Freezing Strategy . . . . .         | 58        |
| <b>5</b> | <b>Experiments and Results</b>                                 | <b>59</b> |
| 5.1      | Pretraining experiments . . . . .                              | 59        |
| 5.1.1    | PaReM pretraining . . . . .                                    | 60        |
| 5.1.2    | TMM-SegRep pretraining . . . . .                               | 61        |
| 5.1.3    | MoCA-T pretraining . . . . .                                   | 62        |
| 5.2      | Classification experiments . . . . .                           | 63        |
| 5.2.1    | Model Variants . . . . .                                       | 63        |
| 5.2.2    | Hyperparameters and Optimization Strategy . . . . .            | 65        |
| 5.2.3    | Evaluation Protocol . . . . .                                  | 65        |
| 5.2.4    | Outcomes . . . . .                                             | 66        |

|          |                                 |           |
|----------|---------------------------------|-----------|
| <b>6</b> | <b>Future Work</b>              | <b>70</b> |
| <b>7</b> | <b>Conclusion</b>               | <b>71</b> |
|          | <b>Acknowledgements</b>         | <b>73</b> |
| <b>A</b> | <b>Configuration Parameters</b> | <b>74</b> |
|          | <b>Bibliography</b>             | <b>79</b> |
|          | <b>List of Figures</b>          | <b>86</b> |
|          | <b>List of Tables</b>           | <b>86</b> |
|          | <b>List of Algorithms</b>       | <b>86</b> |

# List of Abbreviations

|                   |                                                            |
|-------------------|------------------------------------------------------------|
| <b>AMP</b>        | Automatic Mixed Precision                                  |
| <b>BERT</b>       | Bidirectional encoder representations from transformers    |
| <b>ECG</b>        | Electrocardiogram                                          |
| <b>EEG</b>        | Electroencephalogram                                       |
| <b>EMA</b>        | Exponential Moving Average                                 |
| <b>FNN</b>        | Feedforward Neural Network                                 |
| <b>GPU</b>        | Graphics Processing Unit                                   |
| <b>KSS</b>        | Karolinska Sleepiness Scale                                |
| <b>MSE</b>        | Mean Squared Error                                         |
| <b>PaReM</b>      | Patch-Level Reconstruction with Momentum Targets           |
| <b>SSL</b>        | Self-Supervised Learning                                   |
| <b>SSP</b>        | Semantic Subsequence Preserving                            |
| <b>MLP</b>        | Multilayer Perceptron                                      |
| <b>MoCA-T</b>     | Modality-Contrastive Alignment with Temporal Positives     |
| <b>TMM</b>        | Temporal Masked Modeling                                   |
| <b>TMM-SegRep</b> | Temporal Masked Modeling over Segment-Level Representation |
| <b>VICReg</b>     | Variance-Invariance-Covariance Regularization              |

# Chapter 1

## Introduction

Drowsiness detection plays an important role in enhancing traffic safety, especially in the development of adaptive driver monitoring systems. Camera-based methods and behavioral monitoring fall short in terms of early detection and robustness, with poor lighting conditions or occlusion posing notable limitations. Physiological signals such as EEG, ECG, and respiration yield a more objective and fine-grained estimation of the driver’s internal state, capturing subtle cognitive and autonomic changes that precede visual evidence of fatigue. Processing such signals, however, entails unique difficulties, including limited data availability, inter-subject variability of signals, and the lack of objective universal ground truth labels.

This thesis proposes a deep multimodal learning framework that encodes EEG, ECG, and respiratory signals into fused drowsiness representations. In order to combat the absence of reliable labels and generalize better to unseen participants, our approach employs a sequence of chained self-supervised pretraining tasks: Patch-Level Reconstruction with Momentum Targets (PaReM), Temporal Masked Modeling over Segment-Level Representations (TMM-SegRep), and Modality-Contrastive Alignment with Temporal Positives (MoCA-T). Each task introduces a new level of temporal understanding, to effectively model within-segment and across-segment temporal coherence, and aid multimodal alignment under temporal constraints.

The model learns to extract modality-specific features through dedicated unimodal encoders and combines them with a lightweight fusion module. Rather than relying on explicit label regression, the downstream task is redefined as a relative binary classification task, in which the model predicts which of two within-subject multimodal data segments belongs to a drowsier state. This task formulation avoids the subjectivity and in-

consistency of self-ratings while allowing the model to contrast drowsiness levels across a broader spectrum than models focusing only on isolated binary states.

The entire framework follows a modular design, in order to be adjustable to other biosignal-based studies. The model is trained and tested on three data splits, where each split excludes a different set of participants from the training and validation data to allow the evaluation of the model’s generalization capabilities and performance on unseen participants.

By combining multi-level temporal modeling, multimodal fusion, and self-supervised objectives, this thesis offers a scalable method for learning general physiological drowsiness representations, advancing the research on robust fatigue detection.



# Chapter 2

## Literature review

In 2023, the World Health Organization reported that each year around 1.19 million people lose their lives due to road traffic crashes, and another 20 to 50 million people end up with non-fatal injuries. Moreover, for the population aged 5 to 29 years, injuries obtained from traffic accidents are the leading cause of death [1]. Although the estimates of the percentage of fatal traffic accidents related to drowsiness can vary across studies, the American Automobile Association Foundation for Traffic Safety reported this value as 16.5% [2]. These statistics highlight the relevance of drowsiness-related studies, and the development of systems which are capable of drowsiness detection, especially in high-risk environments such as vehicles or industrial settings.

This literature review provides a focused synthesis of prior research and foundational concepts relevant to drowsiness detection, emphasizing the key challenges and architectural design options that informed our decision making process and the development of our proposed model. First, main drowsiness detection methodologies, including behavioral-, performance- and physiological signal-based approaches are introduced. Then different representation learning strategies are compared, from traditional machine learning techniques to modern transformer-based encoders. Next, the review covers the advantages and characteristics of multimodal learning frameworks and possible fusion methods, followed by an overview of commonly applied self-supervised pretraining techniques, such as masked modeling and contrastive learning. Finally, two influential and novel models in this domain are described in detail, which provided inspiration for the architecture proposed in this thesis.

## 2.1 Input Modalities for Drowsiness Detection

As driver drowsiness is a serious risk factor, many attempts have been made to detect dangerous drowsiness levels. The most common methods fell into one of the following three categories: behavioral-based methods, performance-based methods, and physiological-signal-based methods.

- **Behavioral-based methods** rely on observable visual signals such as yawning, gaze direction, eyelid closure, and head nodding [3, 4]. These features are typically extracted using cameras and computer vision models.
- **Performance-based methods** monitor driving behavior, such as how the driver steers the wheel, their lane-keeping patterns, and how fast their response time is to stimuli [5, 6]. These features can be gathered from the vehicle's onboard diagnostics or specialized sensors.
- **Physiological-signal-based methods** aim to directly capture changes in the driver's nervous system by measuring biosignals such as electroencephalography (EEG), electrocardiography (ECG), and respiration [7, 8].

Although behavioral methods are non-invasive and easy to deploy in cars, they are susceptible to lighting conditions, occlusions, individual variability, and the placement of the camera. In addition, visual fatigue signs may appear only in advanced stages of drowsiness, thus being less suitable for early intervention. In case of performance-based methods, driving behavior can be influenced by numerous external factors, including road conditions, traffic, driver distractions and driver skill, making it difficult to isolate drowsiness as the underlying cause. In contrast, while the collection of physiological signals requires more specialized hardware, they offer more objective and more fine-grained assessments of cognitive and physical states. Their ability to reflect internal neurophysiological changes enables earlier detection of drowsiness onset and supports the development of models that generalize more reliably across environments. These properties make them a commonly used input modality in robust drowsiness detection research, and the chosen modality option for this thesis as well.

## 2.2 Representation Learning

Having established the relevance of physiological signals to robust drowsiness detection, the next question is how to process these raw, high-dimensional time-series inputs so that semantically meaningful patterns are retained. This section discusses representation learning methods frequently used in biosignal processing, with the aim of identifying architectures that capture both local signal features and long-range temporal dependencies.

### 2.2.1 Motivation of Drowsiness Representation Learning

One of the fundamental challenges in developing robust drowsiness detection systems is the lack of high-quality, fine-grained, and objective labels. Many currently available models are able to distinguish between a limited number of drowsiness states. Most use only binary labeling (drowsy/wakeful), and their data is purposefully collected in extreme mental states, focusing on severe drowsiness and wakefulness, omitting intermediate states and the differentiation between gradual changes. Other studies heavily rely on self-assessments, like the *Karolinska Sleepiness Scale* (KSS) [9], recorded every few minutes. These labels are temporally sparse, prone to individual variability, and subject to reporting bias, making them unreliable as explicit supervision signals for supervised deep learning models.

This challenge motivates the use of *representation learning techniques*, which aim to extract informative features and compressed latent representations from raw signals, instead of relying on feature engineering. In particular, *unsupervised* and *self-supervised* learning approaches have emerged as promising alternatives to supervised training in domains where label scarcity and subjectivity are major concerns, by uncovering patterns and structure inherent in the data itself.

In the context of drowsiness detection, effective representations should capture temporal patterns and cross-modal dependencies that correlate with cognitive fatigue. Learning such representations has several benefits. It enables the development of models that generalize better across participants, and sessions, while reducing the reliance on subjective human annotations. It makes models more robust to noise through the extraction of high-level features, and allows learned embeddings to be reused in a variety of downstream tasks such as classification, regression, clustering, or change-point detection.

## 2.2.2 Architectures for Representation Learning

### Traditional Machine Learning-Based Approaches

Prior to the emergence of deep learning, traditional machine learning pipelines for processing physiological signals were prone to rely on manually engineered features extracted from the time, frequency, or time-frequency domains. Common features included statistical descriptors (mean, variance), spectral power in specific frequency bands, or entropy-based features. These were then fed into shallow models such as Support Vector Machines (SVMs) [10], K-Nearest Neighbors (KNN) [11] or Fuzzy-Classifiers [12]. While these methods showed reasonable performance in constrained settings, their ability to capture complex temporal relationships or handle inter-participant variability was limited, while these concerns are especially relevant to drowsiness detection.

### Autoencoders

*Autoencoders* are neural networks used in unsupervised representation learning to reconstruct input data from compressed latent representation. They consist of an encoder that maps the input into a lower-dimensional space and a decoder that reconstructs the input. The process makes the network learn salient features that extract the intrinsic structure of the data. Due to their effectiveness in denoising and dimensionality reduction, autoencoders have been utilized in many biosignal processing models [13]. However, they lack the capacity to properly model temporal dependencies, therefore they are better suited for denoising than latent drowsiness representation creation.

### Convolutional and Recurrent Architectures

Later on, *convolutional neural networks* (CNNs)[14] and *recurrent neural networks* (RNNs)[15] became a popular choice for physiological signal processing. CNNs are well adapted to extracting local temporal patterns and have been widely applied for their efficiency and translation invariance [16, 17]. RNNs and their extensions LSTMs [18] have been used to model longer-range temporal dependencies, particularly in EEG and ECG-based systems [19]. These architectures primarily advanced representation learning through enabling end-to-end training from raw signals to output prediction, without manual feature engineering involved.

## Transformer-Based Models

Initially designed for natural language processing applications, *transformer* architectures [20] have since been the foundation of state-of-the-art models across a range of fields including computer vision, audio, and time-series modeling. A central feature of transformers is the *self-attention* mechanism, which enables the model to dynamically decide the significance of various input elements in producing contextualized embeddings.

In the self-attention layer, a weighted sum over all positions in the input sequence is computed for each token or timestep, allowing the model to model long-range dependencies. Since the self-attention mechanism is permutation-invariant, added positional encodings provide information about the relative or absolute location of each timestep, maintaining temporal context.

Transformers are especially appropriate for time series data and other sequential data because they can learn both local and global patterns at the same time. Unlike convolutional models limited by fixed receptive fields, transformers can model arbitrary interactions across time, scaling with sequence length. Since transformers handle all timesteps in parallel, the training time gets significantly reduced compared to concurrent models, making transformers a promising candidate for high-resolution physiological data.

Recent studies have investigated the application of transformers to biosignal processing tasks. For example, the EEGFormer[21] model applied three chained transformers on convoluted EEG signals for an emotion recognition task, while Guang Shi et. al [22] combined an Inception module with a transformer for EEG-based automatic sleep staging. Both outperformed purely CNN and RNN-based variants, highlighting the transformer's superior ability to capture long-range dependencies and global context within sequential physiological data. These models also applied convolutional input embedding layers to extract low-level temporal features and reduce the dimensionality of the raw signals before applying self-attention. Following this architectural trend and the promising results of many similar models [23, 24, 25], our implementation adopts a hybrid approach as well that combines convolutional layers for local feature extraction with transformer encoders for contextual integration, enabling efficient and expressive representation learning from raw physiological inputs.

## 2.3 Multimodal Learning

With a strategy in place for learning expressive representations from single physiological modalities, the next architectural decision concerns whether leveraging multiple modalities can further enhance model performance.

*Multimodal learning* is the process of combining information from various data modalities to construct more expressive, robust, and generalizable models. In human state estimation, multimodal fusion of physiological signals like EEG, ECG, and respiration can significantly enhance performance by taking advantage of complementary observations of the underlying cognitive and physiological states [26].

Each modality measures a distinct attribute of the state of the human body. EEG picks up on direct neural activity and is highly sensitive to cognitive states such as alertness and tiredness [27]. ECG provides information on the functioning of the autonomic nervous system, e.g., heart rate and heart rate variability, which is associated with arousal and stress [28]. Respiration measures breathing rate and patterns, which can be correlated with states of relaxation and alertness [29].

By combining these signals, multimodal systems can compensate for the limitations inherent in each modality. For example, where EEG data is noisy due to sensor malfunction or detachment, ECG or respiratory signals can be used as fallback information. This redundancy holds potential for improving system robustness and generalization across varying conditions [26], therefore a multimodal design was chosen for our architecture.

### Fusion Strategies

One of the most critical aspects of multimodal learning is determining how and when to integrate information between modalities. Fusion methods are usually categorized according to the stage in the model pipeline where the combination occurs. The most common three paradigms are early fusion, middle fusion, and late fusion.

**Early Fusion** is the concatenation of the raw input signals or their low-level features from all modalities before any modality-specific processing [30]. Early fusion treats the inputs as a single joint representation and feeds it into a shared encoder. While simple to implement, early fusion can suffer from scale mismatch, sampling frequency misalignment, and can not capture modality-specific features effectively.

**Intermediate Fusion** (also referred to as representation-level fusion) processes each modality independently with specialized encoders to produce high-level embeddings for each modality. Modality-specific embeddings are then fused—typically by concatenation, attention, or learned projections—into a joint representation. This strategy balances preserving modality-specific features and enabling cross-modal interaction [31, 32]. It is well suited to tasks where modalities contain complementary information but come with different signal dynamics, therefore it is the basis of the approach used in this thesis.

**Late Fusion** operates independently on every modality and integrates the resulting predictions through ensemble techniques, voting, or weighted averages [33]. Even though this method may be robust to missing or noisy modalities, it generally does not take full advantage of cross-modal interactions and will underperform for deep semantic integration.

## 2.4 Self-supervised Learning

After defining the architectural backbone which comprises unimodal encoders and an intermediate fusion strategy for physiological data modalities, the next design decision in this thesis concerned how to best train the model to produce generalizable and semantically meaningful embeddings—especially given the scarcity of labeled data and the inherent subjectivity of drowsiness annotations.

*Self-supervised learning* (SSL) has proven to be a strong paradigm for representation learning in scenarios where the annotated data is limited, expensive to obtain, or noisy. Unlike supervised learning, which is dependent on human-annotated labels, SSL leverages the internal structure and semantics of the data to define pretext tasks. These tasks encourage the model to gain a meaningful understanding of the input without human supervision.

SSL has revolutionized computer vision with methods such as SimCLR [34], MoCo [35], and BYOL [36], and similar ideas have been applied in recent years on time-series and biosignals as well. In physiological domains such as EEG and ECG, where costly expert supervision-based labeling or subjective self-ratings are often inconsistent, SSL offers a viable solution for training robust models [37, 38, 39].

## Motivation for Self-supervision in Physiological Signal Processing

Physiological datasets often exhibit the following characteristics:

- **Sparse labels:** Drowsiness ratings or cognitive state labels are typically recorded at intervals (e.g., every 5 minutes), making fine-grained supervised training difficult.
- **Subjective annotations:** Self-reports like the Karolinska Sleepiness Scale (KSS) [9] are inherently subjective and noisy.
- **Cross-subject variability:** Different individuals exhibit different physiological patterns, even under similar states, making generalization difficult.

By learning to encode the temporal, spatial, and statistical structure of physiological signals through self-supervised objectives, models can acquire features that better generalize across subjects and domains and are more robust to noise. These learned representations can then be fine-tuned on small supervised datasets to better align with the specific use case, or they can be used directly for downstream tasks such as classification or clustering.

### 2.4.1 Masked Modeling

*Masked modeling* is a self-supervised learning technique in which random parts of the input data are deliberately hidden (masked), and the model is trained to reconstruct or predict the missing elements. Unlike autoencoders, which attempt to restore the entire input and may overfit to noise or low-frequency patterns, masked modeling aims for the reconstruction of randomly selected regions only, improving robustness and efficiency. Originally popularized in natural language processing by BERT [40], the technique has since been extended to computer vision (e.g., MAE [41]) and time-series data, including physiological signals [42, 43, 44].

The central idea behind masked modeling is to encourage the model to develop a deep contextual understanding of the input. To predict randomly masked segments accurately, the model must infer their content from the surrounding context, thus learning semantic and temporal dependencies. This is especially advantageous in the context of physiological time-series data, where biosignals come with an inherent sequential structure and can form patterns across multiple input segments.



Recent biosignal studies reflect the benefits of masked pretraining tasks. BENDR [43] uses masked modeling over encoded EEG sequences, contrasting convolved and contextualized representations output by a transformer encoder, while MAEEG[44] aims to reconstruct the raw EEG signal from masked convolved input. However, due to the low signal-to-noise ratio and wide range of amplitudes in EEG, reconstructing raw signals can introduce errors impacting the loss function. Therefore novel approaches, like EEG2Rep [42], focus on reconstruction in the latent space, where abstract features provide normalized and less noise-contaminated targets.

### 2.4.2 EEG2Rep

EEG2Rep [42] is a state-of-the-art self-supervised learning framework designed to extract robust representations from EEG signals without relying on labeled data. It follows the design principles of contrastive and predictive self-supervised methods based on a target and student network such as MoCo [35] and BYOL [36], but adapts them to the temporal and structural properties of EEG recordings. The model is designed to overcome the common challenges in EEG modeling, such as label scarcity, signal noise and hard-to-segment sequences, by leveraging a masked patch prediction task over time-embedded EEG segments instead over raw data.

The EEG2Rep architecture consists of three main components:

- **Context Encoder:** A transformer-based encoder that receives selected parts of the input and generates their contextualized intermediate latent representations using self-attention.
- **Target Encoder:** A momentum-updated encoder identical in its architecture to the context encoder. It processes the whole input sequence, providing target representation for the prediction task.
- **Predictor:** A cross-attention transformer that reconstructs blocks of the target encoder's output based on context representations from the context encoder.

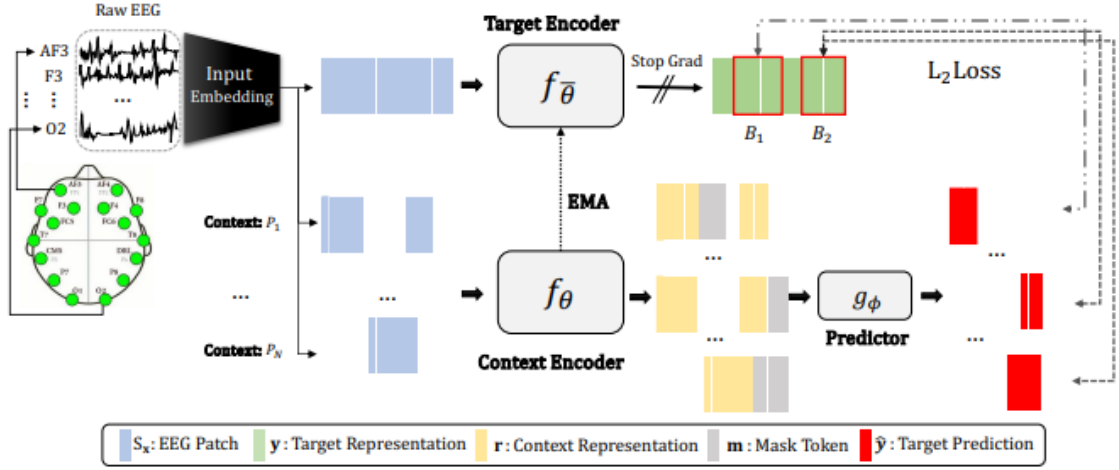


Figure 2.1: EEG2Rep architecture figure taken from the original paper[42]

### Self-supervised Task

The model takes a segment of raw multi-channel EEG data as input, then transforms it into small patch embeddings using a series of depthwise and spatial convolutions. The *target encoder* receives the whole sequence of patch embeddings, and creates semantically rich representations by introducing contextual information through self-attention. Since this allows information encoding over the whole input, these representations serve as targets for the context encoder and predictor during the self-supervised pretraining.

The *context encoder* receives only subsets of the patch embeddings, called contexts, and encodes them into context representations. Random blocks of contextualized patch embeddings are selected from the target representation, and the *predictor* network tries to reconstruct each of them from each context representation through cross-attention between the context embeddings and mask tokens corresponding to the target blocks' patch embeddings. The training objective is to minimize the average L2 loss between the predicted and target representations:

$$\text{Loss}_{\text{rec}} = \frac{1}{M \times N} \sum_{q=1}^N \sum_{i=1}^M \sum_{j \in B_i} \|y_j - \hat{y}_j\|_2^2 \quad (2.1)$$

where  $M$  denotes the number of target blocks,  $N$  is the number of created contexts, while  $B_i$  is a single target block of patch embeddings. The whole process is illustrated by Figure 2.1, taken from the EEG2Rep paper[42].

### Semantic Subsequence Preserving

The contexts are created through a novel masking strategy introduced by the paper, called *Semantic Subsequence Preserving* (SSP). Randomly selecting embeddings to mask out often leads to narrow missing blocks which can be trivially reconstructed e.g. through interpolation. On the other hand, block masking which guarantees longer consecutive masked segments can easily result in visible sequences too short to retain valuable semantic information, hindering the creation of meaningful context representations. To avoid these issues the EEG2Rep framework applies Semantic Subsequence Preserving, which focuses on creating blocks of visible patches instead of masked ones by symmetrically expanding random starting points.

### Momentum Update

The context encoder is trained using backpropagation, while the target encoder's weights are updated only through *exponential moving average* (EMA) from the context encoder weights. This separation helps encourages the model to learn stable representation throughout training. The following equation is used for the update:

$$\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau) \theta \quad (2.2)$$

where  $\bar{\theta}$  denotes the parameters of the target network,  $\theta$  the parameters of the context network, and  $\tau \in [0, 1]$  the coefficient controlling the update rate.

### Variance-Invariance-Covariance regularization

Besides utilizing EMA updates for the target encoder to avoid representation collapse, an additional regularization is employed to encourage the model to learn high-quality representations by promoting variety among the created embeddings. *Variance-Invariance-Covariance Regularization* (VICReg) [45] is applied on the latent embeddings, which introduces a variance and covariance term, making the full training objective consist of three loss components:

- *Invariance*: the previously defined reconstruction loss.

$$\text{Loss}_{\text{rec}} = \frac{1}{M \times N} \sum_{q=1}^N \sum_{i=1}^M \sum_{j \in B_i} \|y_j - \hat{y}_j\|_2^2 \quad (2.3)$$

- *Variance*: encourages each dimension of the embeddings to maintain its variance above a certain threshold across the batch to avoid collapse.

$$v(R) = \frac{1}{d} \sum_{j=1}^d \max(0, \gamma - S(r^j, \varepsilon)), \quad \text{where } S(x, \varepsilon) = \sqrt{\text{Var}(x) + \varepsilon}$$

- *Covariance*: decorrelates the dimensions of the embeddings by penalizing off-diagonal elements in the covariance matrix, promoting expressive features instead of redundant ones.

$$c(R) = \frac{1}{d} \sum_{i \neq j} [C(R)]_{i,j}^2 \quad (2.4)$$

The final loss is the weighted sum of these terms:

$$\text{Total Loss} = \lambda \text{Loss}_{\text{rec}}^* + \mu v(R) + \gamma c(R) \quad (2.5)$$

### 2.4.3 Contrastive Learning

*Contrastive learning* is a self-supervised learning approach that trains a model to distinguish between similar (positive) and dissimilar (negative) pairs of data samples. The core objective is to learn an embedding space where semantically similar samples are pulled closer together, while dissimilar samples are pushed apart, enabling the learning of discriminative features. Instead of relying on explicit labels, supervision is achieved by custom similarity constraints and augmented views of the data.

This technique has been widely adopted in computer vision [34, 46], natural language processing [47, 48], and increasingly, in time-series and biosignal processing. The TS-TCC framework created by Emadeldeen Eldele et al. [49] defines positive pairs by taking temporally close segments of single-channel EEG signals, resulting in improved performance on a sleep stage classification task. Besides temporal dynamics, the spatial structure or multimodal nature of the data can be leveraged as well. The BIOT model [38] creates different views of the same multi-channel biosignal input by randomly dropping out part of its channels, and contrasts the resulting latent representations after passing them through a transformer encoder. The vigilance estimation framework proposed by Meihong Zhang et al. [50] takes advantage of multimodality and creates positive pairs from the embeddings of different modalities (EEG and EOG) belonging to close timestamps.

For the application of conventional contrastive loss functions, such as InfoNCE [51], an anchor-positive pair is defined and compared against a set of negatives for each sample. However, to get enough diverse negative samples, large batch sizes are needed which increases the computational and memory requirements of training. Moreover, in domains like physiological signals, defining what constitutes a negative pair is often ambiguous and unreliable, especially when inter-subject variability is high [52]. Semantically similar samples could be mistakenly identified as negatives based on temporal distance only, since human state changes are subject to fluctuation. Categorizing samples belonging to different participants can also be misleading, since these participants could easily portray the same underlying state, for which we would want our model to produce similar embeddings, and not push them apart. To avoid these issues, many recent models, like CroSSL [39] opt for positive-only contrastive learning, and that is the followed approach in our architecture as well.

#### 2.4.4 CroSSL

*CroSSL* (Cross-modal Self-Supervised Learning) [39] introduces a novel methodology for robust representation learning from multimodal time-series data through self-supervision. CroSSL addresses two critical limitations in earlier self-supervised frameworks: the requirement of negative pair sampling and the inability to handle missing modalities during training or inference.

The central idea of CroSSL is to apply latent masking on the intermediate unimodal embeddings produced by modality-specific encoders. Unlike traditional input-level masking (common in masked autoencoders), latent masking occurs at the representation level and is designed to selectively hide information that might be modality-specific. After concatenation, multiple views of the same sample are created. Either random parts of the embeddings are masked, or the whole embedding of a single modality is replaced by mask tokens. Each differently masked version is then fused by the same cross-modal aggregator to generate their global embeddings. The global representations of the different views of a single sample serve as positive pairs, and they are pulled closer together in the resulting multimodal latent space. This encourages the model to rely more on information shared across modalities, and create embeddings which represent the underlying phenomena (e.g. drowsiness) influencing all of them, instead of focusing on unimodal features.

### Positive-only Contrasting

To avoid the previously mentioned problem of fake negative pair definition easily arising in the domain of physiological signals, CroSSL utilizes positive pairs of representations only, with the objective of minimizing the distance between them. To prevent the collapse of the resulting latent space, caused by the model producing constant embeddings to provide a trivial solution, the framework applies a variant of Variance-Invariance-Covariance Regularization (VICReg) [45]. Similarly to EEG2Rep [42], the loss of CroSSL balances three components:

1. **Variance loss**, which encourages sufficient spread in each dimension of  $Z$  embeddings by imposing a minimum variability threshold  $\gamma$  across the batch;

$$v(Z^{\{1|2\}}) = \frac{1}{D} \sum_{j=1}^D \max \left( 0, \gamma - \delta(z_{i..N,j}^1, \epsilon) \right) \quad \text{where } \delta(z, \epsilon) = \sqrt{\text{Var}(z) + \epsilon}$$

2. **Invariance loss**, which minimizes the distance between paired global embeddings  $Z^1$  and  $Z^2$  (differently masked versions of the same input);

$$s(Z^1, Z^2) = \frac{1}{N} \sum_{i=1}^N \frac{1}{D} \sum_{j=1}^D \|z_{i,j}^1 - z_{i,j}^2\|_2^2 \quad (2.6)$$

3. **Covariance loss**, which punishes off-diagonal elements in the covariance matrix  $C$  to foster more informative features.

$$c(Z) = \frac{1}{D} \sum_{j=1, j \neq j'}^D [C(Z)]_{j,j'}^2 \quad (2.7)$$

### Resilience to Sensor Failures

A key strength of CroSSL is its robustness to missing modalities. During training, random or spatial latent masking simulates the unavailability of sensor channels or data gaps. This forces the aggregator to develop the ability to integrate information from arbitrary subsets of modalities, a property that is especially advantageous in real-world physiological monitoring applications where sensor data may intermittently get corrupted.

## 2.5 Summary

This literature review provided a systematic overview of the research areas behind the proposed method for driver drowsiness detection from multimodal physiological signals. It started by introducing the limitations of traditional behavioral and performance-based methods, which led to the use of physiological signals for more precise and earlier-stage drowsiness detection. Then several representation learning methods were described which are suitable for biosignals, highlighting the advantages of the combined use of convolutional networks and transformers for efficient input embedding.

The review explored the use of multiple modalities and the options for their fusion for more robust and informative representation creation, then discussed commonly applied self-supervised pretraining tasks to enhance generalization and handle label scarcity.

Finally, two recent models—EEG2Rep [42] and CroSSL[39] —were explained in detail to illustrate how the advancements of self-supervised representation learning can be applied in biosignal domains. These models outperformed competitive models such as BENDR [43], BIOT [38], MAEEG [44] and COCOA [53] across various biosignal tasks, including sleep stage detection, driver distraction classification and stress recognition. These promising results motivated the adaptation of modality and temporal masking in our proposed architecture.

In conclusion, the reviewed literature here presents that successful drowsiness detection benefits from a combination of modality-aware representation learning, temporal context modeling, and self-supervised objectives. These insights informed the design and implementation of the proposed multimodal deep learning pipeline detailed in the next chapter.

# Chapter 3

## Methodology

### 3.1 Goals and Motivations

The overarching goal of this thesis is to develop a multimodal deep neural network architecture capable of creating robust and generalizable representations for estimating human drowsiness from physiological time-series signals. Fatigue-related impairments are among the leading causes of accidents in high-risk settings such as driving and industrial monitoring. Early and reliable detection of drowsiness holds substantial potential for enhancing safety and enabling intervention.

#### Challenges

Multiple challenges shape the design and motivation of this work:

**Absence of definitive ground truth:** Drowsiness labels are subjective and typically grounded in self-reports or expert observations, both of which are known to be inconsistent. No universal or objective ground truth exists.

**Single-modality limitations:** Isolated physiological signals, such as EEG or ECG alone, may fail to capture the complex underlying drowsiness state.

**High inter- and intra-subject variability:** Physiological patterns associated with drowsiness vary greatly across individuals and even within individuals over time, making it difficult to learn global rules.

**Limited data availability:** The dataset used in this work contains high-resolution EEG, ECG, and respiratory signals, which are difficult and costly to collect, especially in synchronized fashion, resulting in a small sample size.



**Lack of public datasets and benchmarks:** Due to the costly collection of physiological signals, most such datasets are private. Public datasets are limited in size, modality number, or demographic diversity. To our best knowledge, no public dataset exists with jointly recorded EEG, ECG and respiratory signals in a driving setting, hindering cross-study comparison and benchmarking.

**Lack of pretrained models and :** Unlike domains such as vision or language, there are no publicly available pretrained models on similar biosignals to leverage.

#### **Proposed Solution**

To address these challenges, this thesis proposes a flexible deep learning pipeline that learns rich latent representations of EEG, ECG, and respiratory signals and refines them across multiple self-supervised pretraining stages, each of which captures a different temporal or multimodal aspect of drowsiness. A final classifier is trained on pairwise multimodal samples from the same individual, predicting which of two 1-minute segments corresponds to a drowsier state. This objective avoids the need to regress to inconsistent and subjective absolute self-ratings, while enabling the comparison between drowsiness levels across a broader spectrum.

The architecture is designed to progressively build up semantic understanding from unimodal feature extraction to multimodal fusion and alignment. The central hypothesis is that incorporating pretraining stages of temporal modeling at multiple levels—within unimodal segments, across unimodal embeddings, and among multimodal representations—can guide the model to extract stable, semantically meaningful multimodal latent representations that improve generalization and provide a strong foundation for downstream drowsiness prediction tasks.

The model operates directly on raw time-series data rather than handcrafted features. This allows it to learn complex patterns across time and frequency domains automatically and reduces bias introduced by feature selection. The length of input segments was chosen to be one minute to balance the trade-off between capturing sufficient physiological changes and maintaining a manageable temporal resolution and a higher number of training samples. Shorter segments may miss important fluctuations and patterns, making the representations too granular, while longer ones would drastically reduce the available training instances and increase the memory and computational needs of the model due to the frequent use of Transformer models which scale quadratically with input length.

## 3.2 DrowsiRank Architecture

In this section, the classifier architecture implemented and evaluated in this thesis, referred to as DrowsiRank, is introduced. This model integrates multiple sequentially pretrained modules, each responsible for different stages of feature extraction, fusion, and classification. The architecture encodes paired 1-minute segments of raw synced physiological signals (EEG, ECG and respiration) obtained from the same participant, and predicts which one corresponds to a higher drowsiness state. Figure 3.1 provides an overview of our proposed model.

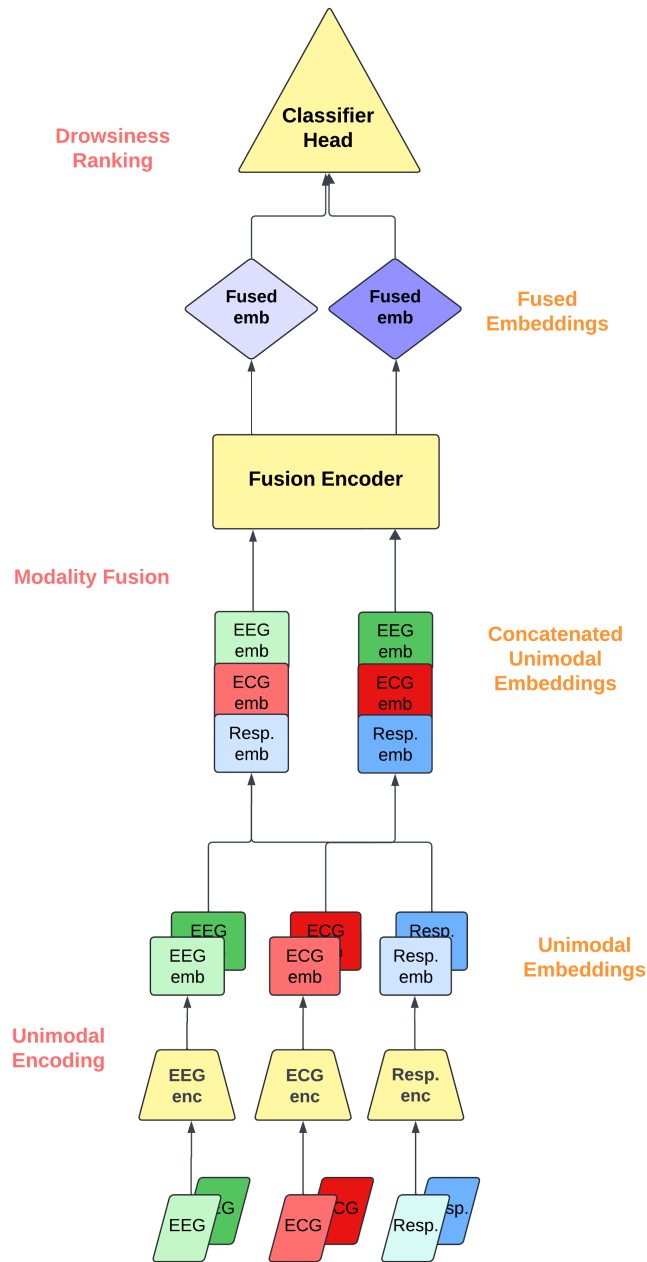


Figure 3.1: DrowsiRank architecture

### 3.3 Model Components

#### 3.3.1 Unimodal Encoders

Each physiological modality—EEG, ECG, and respiration—is processed using its own dedicated unimodal encoder to allow the learning of modality-specific features. These unimodal encoders convert a fixed-length sequence of raw physiological signals into dense representations of unimodal embeddings, extracting robust features. The encoders output a single embedding per segment, capturing both temporal and spatial patterns from the raw signal.

The encoder applies three main transformations: input embedding, optional sequence encoding, and representation pooling. The steps of a single forward pass is detailed in Algorithm 1.

##### Input Embedding via Patching

The raw signal has a shape of  $(B, C, T)$ , where  $B$  is batch size,  $C$  is the number of channels and  $T$  is the temporal sequence length of a sample. The input is first transformed into a sequence of fixed-length patches suitable for transformer-based processing, by using a convolutional embedding module. The module includes:

- A depthwise convolution to extract channel-wise features and project to the embedding space;
- Multiple spatialwise convolutions to capture local temporal dependencies;
- Downsampling and max pooling to reduce sequence length and create fixed-size patches;
- Optional additional convolutional layers for further reduction and refinement.

The output of this module is a sequence of patch embeddings which encode local patterns in the signal while preserving its temporal nature with shape  $(B, P, D)$ , where  $B$  is batch size,  $P$  is the number of patches, and  $D$  is the embedding dimension.

### Transformer-Based Contextual Encoding

The sequence of patch embeddings is then passed through a Transformer encoder stack referred to as the *context encoder*. Sinusoidal positional encodings are added to retain temporal order. Each Transformer block consists of multi-head self-attention, feed-forward layers with GELU activation, residual connections and layer normalization. This component is responsible for modeling contextual interactions between signal patches.

For specific ablation experiments or efficiency constraints, the architecture also supports a *convolution-only* mode, in which the Transformer layers are disabled, and the patch embeddings received from the input embedding module are passed directly to the output head.

### Output Embedding

For downstream tasks, the output of the Transformer (or embedding module in convolution-only mode) is passed through a global average pooling layer over the patch dimension. This yields a unimodal embedding, a single fixed-size vector per 1-minute segment, serving as the input for later model components.

---

**Algorithm 1** Forward pass of the Unimodal Encoder

---

**Require:**  $x$ : input signal of shape  $(B, C, T)$   
**Ensure:**  $z$ : latent embedding of shape  $(B, D)$

```

1:  $p \leftarrow \text{InputEmbedding}(x)$  ▷ Output shape  $(B, P, D)$ 
2:  $p \leftarrow \text{LayerNorm}(p)$ 
3: if not conv_only then
4:    $p \leftarrow p + \text{PositionalEncoding}(p)$ 
5:    $p \leftarrow \text{LayerNorm}(p)$ 
6:    $p \leftarrow \text{ContextEncoder}(p)$ 
7: end if
8:  $p \leftarrow \text{Transpose}(p, \text{shape}(B, D, P))$ 
9:  $z \leftarrow \text{GlobalAvgPool}(p)$  ▷ Average over time/patch dimension
10: return Squeeze( $z$ ) ▷ Final shape  $(B, D)$ 

```

---

### 3.3.2 Fusion Encoder

The fusion encoder is responsible for integrating the unimodal embeddings created by the unimodal encoders into a fused latent representation that captures the shared information between modalities. In this work, we use a fully-connected feedforward neural network (FNN) as the fusion encoder. Specifically, unimodal embeddings—each of size  $D$ —are concatenated to form a single vector of dimensionality  $M \cdot D$ , where  $M$  is the number of modalities. This concatenated vector is then passed through a series of linear layers with ReLU activations, progressively reducing the dimensionality to match the desired latent space. In our implementation, the FNN has the following architecture:  $\text{Linear}(M \cdot D \rightarrow 2 \cdot D) \rightarrow \text{ReLU} \rightarrow \text{Linear}(2 \cdot D \rightarrow 2 \cdot D) \rightarrow \text{ReLU} \rightarrow \text{Linear}(2 \cdot D \rightarrow D)$ . The output of the fusion encoder is a single fused representation of dimensionality  $D$ , which serves as the input to downstream modules such as the classification head during fine-tuning. This simple yet effective fusion strategy allows the network to learn meaningful interactions between modalities while keeping the architecture lightweight and interpretable. We opted for intermediate fusion because it first enables the extraction of modality-specific features, which especially in the domain of noisy and highly variable physiological signals is essential. Additionally, unlike late fusion, intermediate fusion makes the interaction and discovery of relations between modalities possible, which serves our goal of creating multimodal embeddings representing the information intersection of modalities.

### 3.3.3 Classifier Head

The final classification module, referred to as the classifier head, is responsible for predicting which of two input samples corresponds to a higher level of drowsiness. Given two embeddings  $\mathbf{z}_i$  and  $\mathbf{z}_j$  obtained from the embedding backbone (comprised of the unimodal encoders and the fusion encoder), the classification head first combines them using either vector concatenation  $[\mathbf{z}_i; \mathbf{z}_j]$  or directional difference  $(\mathbf{z}_i - \mathbf{z}_j)$ , depending on the selected configuration. In our experiments, we primarily used the directional difference mode to highlight relative changes in drowsiness, and encourage the model to create a structured latent space with separable embedding groups corresponding to different drowsiness states.

The resulting combined representation is then passed through a two-layer feedforward network. The first layer reduces the embedding size to a hidden dimension of  $h$  (typi-

cally  $D/2$ ), followed by a ReLU activation and optional dropout for regularization. The second layer projects the features into the final output space of size two, corresponding to a binary classification task. Formally, the classification head follows the architecture:  $\text{Linear}(M \cdot D \rightarrow h) \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(h \rightarrow 2)$ . The output logits are then interpreted as class scores to determine which of the two samples is more likely to exhibit higher drowsiness. This pairwise comparison framework encourages the model to develop sensitivity to fine-grained differences in latent drowsiness levels, making it suitable for learning personalized or temporally-aware ranking patterns.

### 3.4 Training Pipeline

Self-supervised learning relies on the design of an appropriate pretraining task - an auxiliary objective that encourages the model to extract meaningful representations from the data. To better capture the temporal structure of physiological signals, we propose a multi-stage pretraining framework that extends the CroSSL [39] design by incorporating temporal modeling at every level of the architecture:

- Each unimodal encoder is pretrained using a self-prediction task inspired by EEG2Rep [42] to model temporal structure within individual segments;
- These encoders are further fine-tuned via temporal masked modeling over consecutive multi-minute data sequences to leverage inter-segment temporal information;
- Besides contrastive learning based on modality masking, the training objective of the fusion model incorporates temporal contrasting by treating segments within a defined local temporal window as positive pairs as well.

### 3.4.1 Patch-Level Reconstruction with Momentum Targets (PaReM)

The Patch-Level Reconstruction with Momentum Targets (PaReM) stage trains modality-specific encoders based on patch-level masked modeling and representation alignment. We adopt the key idea from the EEG2Rep framework [42], using transformer-based momentum-updated target encoder, a context encoder and a predictor to reconstruct randomly masked patch embeddings.

#### Self-Supervised Masked Modeling

The unimodal encoders are pretrained using a masked modeling objective inspired by EEG2Rep [42]. During training, a subset of the patch embeddings received from the input embedding module is selected as masked elements following the semantic subsequence preserving strategy to retain temporally meaningful visible subsequences. The visible subset is passed through the context encoder, while the full (unmasked) input is processed by a target encoder—a momentum-updated exponential moving average (EMA) copy of the context encoder.

A cross-attention-based predictor then attempts to reconstruct the target encoder’s output embeddings corresponding to the masked positions, using the context representation (the output of the context encoder for the visible patches) and a learnable mask token. The steps of a single PaReM forward pass is detailed in Algorithm 2.

#### Momentum Target Network

The target encoder is not updated via backpropagation, instead, its parameters are updated using a momentum-based exponential moving average of the context encoder. This makes the weights of the target encoder represent a delayed and more stable version of the weights of the context encoder. By smoothing out fluctuations in the encoder parameters, the target encoder provides consistent guidance for prediction, helping the model focus on learning generalizable structure in the data rather than overfitting to transient patterns. This setup promotes more robust representation learning and mitigates the risk of collapse to trivial solutions. The momentum coefficient is linearly scheduled from  $\tau_0$  to  $\tau_e$  during a warm-up period, as in BYOL [36].

---

**Algorithm 2** Forward pass of the Unimodal Encoder during PaReM
 

---

**Require:**  $x$ : input signal tensor of shape  $(B, C, T)$

**Ensure:**  $[\hat{r}_{\text{masked}}, r_{\text{masked}}, r_{\text{context}}, r_{\text{target}}]$

```

1:  $p \leftarrow \text{InputEmbedding}(x)$  ▷ Shape  $(B, P, D)$ 
2:  $p \leftarrow \text{Norm}(p)$ 
3:  $p \leftarrow p + \text{PositionalEncoding}(p)$ 
4:  $p \leftarrow \text{Norm2}(p)$ 
5:  $m \leftarrow \text{RepeatMaskToken}(B, P)$ 
6:  $m \leftarrow m + \text{PositionalEncoding}(m)$ 
7:  $I \leftarrow \text{Range}(P)$ 
8:  $\text{visible\_idx} \leftarrow \text{SemanticPreserving}(I, \text{block\_count}, \text{mask\_ratio})$ 
9:  $\text{mask\_idx} \leftarrow I \setminus \text{visible\_idx}$ 
10:  $p_{\text{visible}} \leftarrow p[:, \text{visible\_idx}, :]$ 
11:  $m_{\text{token}} \leftarrow m[:, \text{mask\_idx}, :]$ 
12:  $r_{\text{context}} \leftarrow \text{ContextEncoder}(p_{\text{visible}})$ 
13:  $r_{\text{target}} \leftarrow \text{TargetEncoder}(p)$  ▷ No gradient update
14:  $r_{\text{masked}} \leftarrow r_{\text{target}}[:, \text{mask\_idx}, :]$ 
15:  $\hat{r}_{\text{masked}} \leftarrow \text{Predictor}(r_{\text{context}}, m_{\text{token}})$ 
16: return  $[\hat{r}_{\text{masked}}, r_{\text{masked}}, r_{\text{context}}, r_{\text{target}}]$ 
    
```

---

### Training objective

The training objective incorporates a VICReg-inspired [45] loss that combines alignment, variance and covariance regularization. Standard MSE loss is calculated between the predicted masked patch embeddings and the corresponding target embeddings. Then to avoid representation collapse and aid the creation of diverse and meaningful representations forming a well-structured embedding space, covariance loss and a feature-wise variance loss gets calculated across the produced predictions in the batch. The final loss is the sum of these components. The exact calculation process is shown by Algorithm 3.

---

**Algorithm 3** Loss Computation for Patch-Level Reconstruction
 

---

```

1: function CALCULATELOSS( $\hat{r}, r$ ) ▷  $\hat{r}$ : predicted representations,  $r$ : target representations
2:    $L_{\text{align}} \leftarrow \text{MSE}(\hat{r}, r)$  ▷ Compute patch-wise alignment loss
3:    $Y \leftarrow \text{GlobalAveragePooling}(\hat{r})$ 
4:    $Y \leftarrow Y - \text{Mean}(Y, \text{axis} = 0)$  ▷ Center features by removing batch mean
5:    $\sigma \leftarrow \sqrt{\text{Var}(Y, \text{axis} = 0) + \epsilon}$ 
6:    $L_{\text{std}} \leftarrow \text{Mean}(\text{ReLU}(1 - \sigma))$  ▷ Encourage unit variance for each feature
7:    $C \leftarrow \frac{Y^T Y}{N-1}$ 
8:    $L_{\text{cov}} \leftarrow \text{Mean}(\text{OffDiagonal}(C)^2)$  ▷ Minimize feature covariance
9:   return  $L_{\text{align}}, L_{\text{std}}, L_{\text{cov}}$ 
10: end function
    
```

---



### Benefits of Patch-Level Pretraining

Patch-level reconstruction pretraining allows each unimodal encoder to learn fine-grained temporal structure within physiological signals by reconstructing masked signal patches from their surrounding context. Since each patch corresponds to a short time interval, the model is encouraged to capture local temporal dependencies—such as the dynamics of EEG waveforms or the morphology of cardiac cycles— from raw input. In the context of drowsiness representation learning, this enables the encoders to identify and internalize subtle, temporally localized patterns that may correlate with the onset or progression of fatigue. By doing so in an unsupervised manner, the encoders develop robust, context-aware features that serve as a strong initialization for downstream stages of the pipeline, especially when generalizing across participants and in settings with limited labeled data.

### 3.4.2 Temporal Masked Modeling over Segment-Level Representations (TMM-SegRep)

To enhance the temporal coherence of unimodal embeddings across consecutive signal segments, the Temporal Masked Modeling over Segment-Level Representations (TMM-SegRep) pretraining stage forms the second step of the proposed pipeline. Inspired by BERT-style [40] pretraining, its aim is to fine-tune the unimodal encoders pretrained through the PaReM stage to produce embeddings suitable for capturing higher-level temporal dynamics and contextual dependencies across multiple minutes of data.

#### Input and Preprocessing

The training samples for this stage are windowed consecutive 1-minute segments of a single physiological signal recorded from the same participant, forming an input tensor of shape  $(B, W, C, T)$ , where  $B$  is the batch size,  $W$  is the number of segments forming a windowed sample,  $C$  is the number of channels, and  $T$  is the time series length of a segment. Each segment is first fed to the corresponding pretrained unimodal encoder to produce its unimodal embedding. This produces a sequence of embeddings of shape  $(B, W, D)$ , where  $D$  is the embedding dimension.

### Temporal Masking Strategy

A predefined fraction (e.g., 50%) of the unimodal embeddings is randomly selected, saved for later comparison, then replaced by a learnable *mask token*. For each sample multiple masked versions are generated, thereby augmenting the effective number of training samples that the model encounters. Unlike patch-level masking in the previous stage, masking in TMM-SegRep operates at the segment level, allowing the model to learn relationships between temporally adjacent segments over a longer timescale. To encode the temporal order of the embeddings, sinusoidal positional encoding is applied to them.

### Transformer-Based Modeling and Reconstruction

The masked embedding sequence is fed to a Transformer encoder [20], which integrates contextual information across the sequence, enabling the model to infer the content of masked segments from their temporal neighbors. Lastly, a lightweight multilayer perceptron prediction head attempts to reconstruct the original segment embeddings from the output of the Transformer encoder at the masked positions. The detailed TMM process is shown in Algorithm 4.

### Loss Function

Since the temporal masked modeling task operates on the representation level rather than raw data, additional safeguards are required to prevent representation collapse and to discourage the unimodal encoder from producing constant, uninformative outputs. While such trivial representations may minimize the masking loss—by enabling easy reconstruction of masked embeddings—they fail to capture meaningful information, directly opposing the objectives of this thesis. To address this, the training objective combines mean squared error (MSE) reconstruction loss at masked positions with VICReg-style variance and covariance regularization terms [45] computed on the produced unimodal embeddings. These components are weighted and summed to form the total loss:

$$\mathcal{L}_{\text{total}} = \lambda_{\text{mask}} \cdot \mathcal{L}_{\text{MSE}} + \lambda_{\text{var}} \cdot \mathcal{L}_{\text{var}} + \lambda_{\text{cov}} \cdot \mathcal{L}_{\text{cov}}$$

where  $\lambda_{\text{mask}}$ ,  $\lambda_{\text{var}}$ , and  $\lambda_{\text{cov}}$  are coefficient weights for the respective terms.

### Temporal Context Length Selection

In this thesis, 16-minute windows were created and utilized as training samples for the TMM-SegRep task. This length was chosen to provide enough context for the model to learn longer-term trends in drowsiness-related signal dynamics, while avoiding overly fragmenting the already limited dataset, and respecting GPU memory and model capacity constraints.

### Objective

By reconstructing missing embeddings based on their temporal neighbors, TMM-SegRep encourages the learning of temporally aligned embeddings that generalize better in downstream tasks such as drowsiness classification. Through this context-based reconstruction, the encoder becomes more sensitive to the progression of physiological states, such as drowsiness, over time.

---

#### Algorithm 4 TMM-SegRep Pretraining Process

---

**Require:**  $x$ : input tensor of shape  $(B, W, C, T)$  (batch of  $W$  1-minute segments)

**Ensure:**  $\mathcal{L}_{\text{TMM}}$ : total loss (masking + regularization)

```

1:  $x \leftarrow x.\text{reshape}(B \cdot W, C, T)$  ▷ Flattening windows
2:  $z \leftarrow \text{Encoder}(x)$ 
3:  $z \leftarrow z.\text{reshape}(B, W, D)$ 
4:  $(\mathcal{L}_{\text{std}}, \mathcal{L}_{\text{cov}}) \leftarrow \text{CALCULATEREGLOSS}(z)$ 
5:  $L_{\text{total}} \leftarrow 0$ 
6: for  $i = 1$  to  $N_{\text{masks}}$  do
7:    $M \leftarrow \text{RandomMaskIndices}(B, W, r)$  ▷  $r$ : mask ratio
8:    $T \leftarrow z[M]$  ▷ Ground truth embeddings at masked positions
9:    $\tilde{z} \leftarrow \text{InsertMaskTokens}(z, M)$ 
10:   $\tilde{z} \leftarrow \text{PositionalEncoding}(\tilde{z})$ 
11:   $\hat{z} \leftarrow \text{TransformerEncoder}(\tilde{z})$ 
12:   $\hat{T} \leftarrow \text{MLPPredictor}(\hat{z}[M])$  ▷ Predicted embeddings
13:   $L_{\text{total}} \leftarrow L_{\text{total}} + \text{MSELoss}(\hat{T}, T)$ 
14: end for
15:  $\mathcal{L}_{\text{mask}} \leftarrow L_{\text{total}} / N_{\text{masks}}$ 
16: return  $\lambda_{\text{mask}} \cdot \mathcal{L}_{\text{mask}} + \lambda_{\text{std}} \cdot \mathcal{L}_{\text{std}} + \lambda_{\text{cov}} \cdot \mathcal{L}_{\text{cov}}$ 

```

---

### 3.4.3 Modality-Contrastive Alignment with Temporal Positives (MoCA-T)

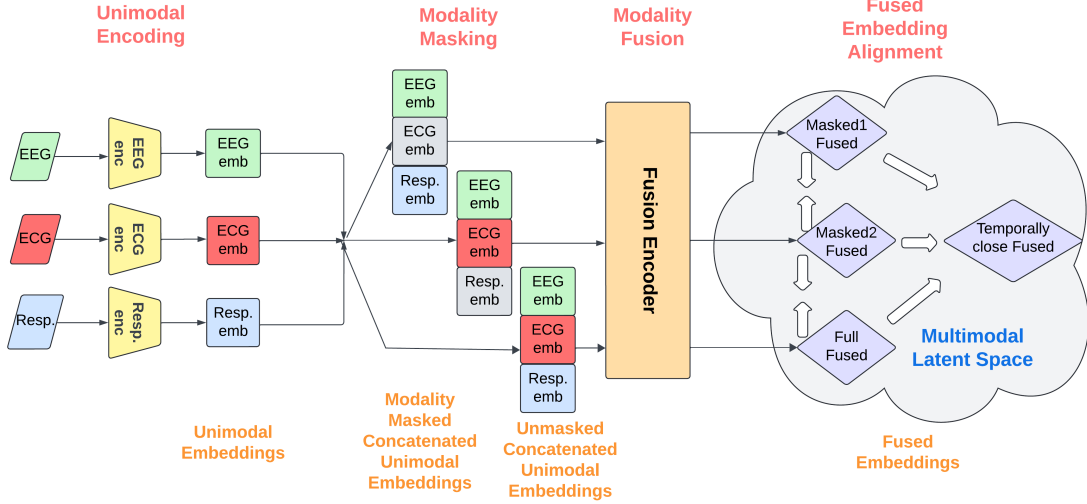


Figure 3.2: Process of MoCA-T Pretraining: unimodal encoding, followed by the fusion of modality masked views and embedding contrasting in latent space. (Grey embeddings symbolize the masked modality.)

In the third pretraining stage, Modality-Contrastive Alignment with Temporal Positives (MoCA-T), a modality masking strategy inspired by CroSSL [39] is applied, along with an additional temporal alignment objective. The goal is to encourage the model to focus during modality fusion on the information intersection of modalities and their underlying shared phenomena, such as drowsiness, instead of modality-specific noise. Moreover, temporal consistency of the fused embeddings is promoted by pulling the representation of temporally neighboring segments closer together in the multimodal latent space. Algorithm 6 details the steps of this pretraining stage.

#### Training Input and Architecture

The input to the MoCA-T stage consists of synchronized multimodal segments, where each modality (in our experiments EEG, ECG and RSP) is passed through its respective unimodal encoder pretrained by the PaReM and TMM-SegRep stages. The  $D$ -dimensional unimodal embeddings are concatenated along the feature dimension to form a single vector per segment of size  $M \cdot D$ , where  $M$  is the number of modalities. This concatenated representation is then processed by the fully-connected fusion encoder, resulting in a fused embedding in the multimodal space.

### **Modality Masking Strategy**

To encourage the model to produce fused embeddings that capture the shared information between modalities rather than feature-specific features, modality-wise masking is applied. Specifically, two randomly masked views of the unimodal embeddings are created per sample. In each view, one of the modalities is completely replaced by a modality-specific learnable mask token. Using learnable mask tokens, rather than simply zeroing out modalities, allows the model to distinguish between intentionally missing inputs and actual low-activation signals. The masked embeddings, along with the version in which each modality is visible, are then passed through the fusion encoder to generate three corresponding latent vectors. This process is illustrated by Figure 3.2.

Alignment loss is calculated for these latent representations by computing the mean squared distance between each possible pair combination, including masked-to-masked and masked-to-unmasked, encouraging the model to learn modality-invariant representations and promote a more coherent and unified latent space. This aligns with our hypothesis that the multimodal latent space should capture robust, high-level features that are resilient to noise, which is a highly beneficial quality for physiological signal processing.

Unlike the original CroSSL architecture [39], which was primarily designed for scenarios involving missing modalities, our model also fuses and contrasts a fully visible view with the masked ones. This enables the model to learn how to handle the joint presence of all modalities which aligns with the inference-time conditions of our use case.

### **Temporal Contrastive Objective**

Beyond aligning modality views, MoCA-T introduces a third level of temporal modeling into the architecture by expecting temporal coherence in the multimodal latent space. This is achieved by contrasting the fused representations of temporally adjacent segments obtained from the same participant. In our implementation, a 10-minute temporal radius was chosen for defining positive pairs, based on the assumption that drowsiness evolves gradually rather than abruptly, therefore a smoother transition between the latent representations is desirable.

### Positive-Only Contrastive Learning

Our method relies on positive-only contrastive learning, which enables the architecture to avoid contrastive learning’s dependence on negative samples — a known source of instability in physiological time series [52]. In drowsiness detection tasks, where samples are often similar due to limited class-variance and gradual changes in state, and where data augmentation or synthetic negatives are often unreliable, this feature is especially valuable. Unlike contrastive losses that may mistakenly push apart fake negatives, e.g. semantically similar segments belonging to different participants or segments with a larger temporal distance between them, our proposed method focuses entirely on aligning semantically meaningful content.

Specifically, the fused unmasked embeddings, along with the embeddings of masked views in the whole batch, are all stacked, and a mean squared distance loss is computed over this set between embeddings considered as positive pairs. Positive segment pairs are defined based on temporal proximity signaled by the segment index and the segments’ corresponding participant ID. This also means that differently masked views of a segment form positive pairs with each other, as well as with masked or unmasked views of temporally neighboring segments. However, each segment was explicitly excluded from being paired with itself, as such self-pairs would trivially satisfy the contrastive objective. Algorithm 5 demonstrates the positive pair identification and loss calculation process.

---

#### Algorithm 5 Temporal Contrastive Loss

---

**Require:**  $E, E_1, E_2 \in \mathbb{R}^{N \times D}$ : original and two masked embeddings

**Require:**  $I \in \mathbb{Z}^N$ : segment indices,  $P \in \mathbb{Z}^N$ : participant IDs,  $r$ : temporal radius

**Ensure:**  $\mathcal{L}_{\text{mse}}$ : contrastive loss,  $N_{\text{pos}}$ : number of positive pairs

---

```

1:  $E_{\text{all}} \leftarrow$  concatenate  $E, E_1, E_2$  along batch dimension ▷  $3N \times D$ 
2:  $I_{\text{all}} \leftarrow$  repeat  $I$  three times
3:  $P_{\text{all}} \leftarrow$  repeat  $P$  three times
4:  $T_{ij} \leftarrow |I_{\text{all}}[i] - I_{\text{all}}[j]|$  ▷ Compute time difference matrix
5:  $S_{ij} \leftarrow P_{\text{all}}[i] == P_{\text{all}}[j]$  ▷ Compute participant identity matrix
6:  $M_{ij} \leftarrow (T_{ij} \leq r) \wedge S_{ij}$  ▷ Positive mask
7:  $M_{ii} \leftarrow 0$  ▷ Remove self-pairs
8:  $N_{\text{pos}} \leftarrow \sum M / 2 + \epsilon$  ▷  $\epsilon$ : small constant
9:  $(i, j) \leftarrow$  all index pairs from upper triangle of  $M$ , excluding diagonal
10:  $L_{ij} \leftarrow M[i, j] \cdot \frac{1}{D} \sum_d (E_{\text{all}}[i, d] - E_{\text{all}}[j, d])^2$ 
11:  $\mathcal{L}_{\text{mse}} \leftarrow \frac{1}{N_{\text{pos}}} \sum L_{ij}$ 
12: return  $\mathcal{L}_{\text{mse}}, N_{\text{pos}}$ 
    
```

---

### Combined Loss Function

To prevent representation collapse, a variant of variance-invariance-covariance regularization (VICReg) [45] is applied to the embeddings. The overall loss function used to train the model in the MoCA-T stage is a weighted sum of this regularization loss and the positive-only contrastive loss:

$$\mathcal{L}_{\text{MoCA-T}} = \lambda_{\text{VICReg}} \cdot \mathcal{L}_{\text{VICReg}} + \lambda_{\text{contrastive}} \cdot \mathcal{L}_{\text{contrastive}}$$

where  $\mathcal{L}_{\text{VICReg}}$  includes the averaged variance and covariance loss computed over the created fused embeddings in the batch by masking type. This dual objective encourages both inter-modal consistency and temporal continuity in the learned representations while demanding non-trivial uncollapsed solutions, which equips the downstream classifier with more informative and robust features.

---

#### Algorithm 6 MoCA-T Pretraining

---

**Require:** modality inputs  $\{x_m\}$ , sample indices  $I$ , participant IDs  $P$

**Ensure:** VICReg and contrastive losses

```

1: function FUSIONPRETRAIN( $\{x_m\}, I, P$ )
2:   Initialize  $Z \leftarrow []$ 
3:   for  $m$  in modality_order do
4:      $z_m \leftarrow \text{Encoder}_m(x_m)$ 
5:     Append  $z_m$  to  $Z$ 
6:   end for
7:    $z_{\text{cat}} \leftarrow \text{Concat}(Z, \text{dim} = -1)$ 
8:    $(\tilde{z}_1, \tilde{z}_2) \leftarrow \text{APPLYMODALITYMASKING}(z_{\text{cat}})$ 
9:    $r_1 \leftarrow \text{FusionEncoder}(\tilde{z}_1)$ 
10:   $r_2 \leftarrow \text{FusionEncoder}(\tilde{z}_2)$ 
11:   $r \leftarrow \text{FusionEncoder}(z_{\text{cat}})$ 
12:   $(\mathcal{L}_{\text{std}}, \mathcal{L}_{\text{cov}}) \leftarrow \text{VICReg}(r_1, r_2, r)$ 
13:  if loss_type = "temporal" then
14:     $(\mathcal{L}_{\text{cont}}, N_{\text{pos}}) \leftarrow \text{TEMPORALMSELOSS}(r, r_1, r_2, I, P)$ 
15:  else if loss_type = "exact_match" then
16:     $(\mathcal{L}_{\text{cont}}, N_{\text{pos}}) \leftarrow \text{EXACTMATCHMSELOSS}(r, r_1, r_2, I, P)$ 
17:  else
18:    Raise error
19:  end if
20:  return  $(\lambda_{\text{std}} \cdot \mathcal{L}_{\text{std}}, \lambda_{\text{cov}} \cdot \mathcal{L}_{\text{cov}}, \lambda_{\text{cont}} \cdot \mathcal{L}_{\text{cont}}, N_{\text{pos}})$ 
21: end function

```

---

### 3.4.4 Classification

The final stage of the proposed pipeline is the classification task which builds on top of the pretrained unimodal encoders and fusion encoder. Each training sample consisted of a pair of 1-minute multimodal segments drawn from the same participant but labeled with different drowsiness levels. For each segment, EEG, ECG, and respiration data were separately processed by their respective unimodal encoders, and the resulting embeddings were concatenated and passed through the pretrained fusion encoder to obtain latent representations. These representations were then compared using either concatenation or element-wise difference, as specified by a configurable combination mode. The resulting vector was passed through the lightweight classifier head to predict which of the two input segments was drowsier. During training, categorical cross-entropy loss was minimized. This formulation enables a ranking-style comparison that reflects the relative alertness of the inputs rather than absolute score prediction, which is advantageous given the limited granularity and unreliability of subjective drowsiness ratings. Algorithm 7 provides a high-level overview of the processing steps of the full DrowsiRank architecture.

---

**Algorithm 7** Forward pass of the DrowsiRank classifier

---

**Require:**  $batch_i, batch_j$ : dictionaries containing modality-wise tensors (e.g., EEG, ECG, RSP) for the first and second segments of each classification pair; each tensor has shape  $(B, C, T)$

**Ensure:**  $y$ : output logits of shape  $(B, \text{num\_classes})$

```

1:  $combined\_batch \leftarrow \{k : \text{Concat}(batch_i[k], batch_j[k], \text{dim} = 0) \text{ for each key } k\}$ 
2:  $combined\_features \leftarrow \text{Backbone}(combined\_batch)$   $\triangleright \text{Shape } (2B, D)$ 
3:  $z_i, z_j \leftarrow \text{Split}(combined\_features, \text{at } B)$ 
4: if combination = "concat" then
5:    $z \leftarrow \text{Concat}(z_i, z_j, \text{dim} = -1)$ 
6: else
7:    $z \leftarrow z_i - z_j$ 
8: end if
9:  $y \leftarrow \text{Classifier}(z)$ 
10: return  $y$ 

```

---



## 3.5 Data Preprocessing

### 3.5.1 Data Characteristics

The thesis builds on a multimodal anonymized dataset collected at Robert Bosch Ltd./Bosch Group, Hungary during a previous project, as part of a broader effort to explore driver monitoring systems in simulated environments. A driving simulator was constructed, with a car seat, a standard steering wheel and pedals, surrounded by multiple high-resolution monitors arranged in a semicircular layout to imitate a realistic driving experience. Participants were instructed to engage in an extended driving task ranging between one and three hours. The virtual environment was powered by the "Shutoko Revival Project"[54], a modded driving scenario which is known for its repetitive and low-stimulus design. This environment was intentionally chosen to induce a monotonous driving experience, which is known to accelerate the onset of fatigue and drowsiness, thereby enabling the observation of physiological changes associated with reduced alertness.

To monitor drowsiness and fatigue throughout the driving session, several physiological sensors were used. A dry-EEG device captured brain activity using eight dry electrodes placed on the scalp based on a fixed layout. Another device recorded single-lead ECG signals and respiratory activity was recorded as well with a respiratory band placed across the chest of participants. Each physiological signal was recorded at 256 Hz. Every five to fifteen minutes, the participants were asked to report their drowsiness level following the Karolinska Sleepiness Scale (KSS) ratings [9], which is often used as 9-graded subjective annotations of drowsiness levels. Measurements were stored separately for each modality in .csv files, collected in folders per participant.

All participants involved in the study were male, which introduces a demographic bias into the dataset. While this was a practical constraint during data collection, it limits the generalizability of the trained models to more diverse populations. In total, recordings were collected from 47 individuals, resulting in approximately 110 hours of multimodal physiological data.

### **3.5.2 Data Cleaning and Transformation**

The raw physiological signals collected from the simulator drives were processed through a comprehensive data cleaning and transformation pipeline to ensure reliability, alignment, and model compatibility. The goal was to remove corrupted samples, synchronize modalities, apply physiological filtering, prepare the data into well-defined one-minute training samples and create global datasets which contain the samples from all participants.

#### **Management of Corrupted and Misaligned Sensor Data**

Each recorded session was subjected to strict inspection to ensure proper data quality for the model. A subset of the sessions was discarded due to excessive noise, inconsistent sampling rate, sensor failure, missing data or other irregularities. These included drives where one or more modalities were too short, had prolonged periods of missing samples, or were clearly misaligned with other modalities. A manually curated list was maintained to exclude these participants from further multimodal processing. However, in order to fully exploit the available data and minimize data loss from the already limited dataset, when possible, valid modalities were kept for the pretraining tasks that depend only on a single modality. This way, 36 sessions were maintained for the multimodal dataset, which gives 4645 1-minute samples, while 38 drives were usable for ECG, 43 for EEG, and 44 for respiration. In some cases, only a part of the drive was kept, eliminating noisy segments from the beginning or end of the session. The handling of unimodal and multimodal data differs in these scenarios as well, for unimodal these truncations only affected the problematic modality, while for multimodal the corresponding segments from all modalities got eliminated as well.

#### **Synchronization and Segmentation**

All signals — EEG, ECG, respiration, and self-ratings — were aligned in time for each participant. A few sessions had differing start times for the modalities, those got manually aligned first, then all modalities were truncated to the length of the shortest recording among them.

After synchronization, all physiological signals were segmented into non-overlapping windows of one minute in duration. Each segment corresponds to  $60 \text{ seconds} \times 256 \text{ Hz} = 15,360$  samples per channel. This resulted in segment shapes of:

- **EEG:** (15360, 8)
- **ECG and Respiration:** (15360, 1)

Each one-minute segment was assigned the closest drowsiness label in time from the reported KSS self-ratings of the participant based on their timestamp. As meta-data, the ID of the participant and the segment's relative index within the measurement got stored as well, which not only serve validation and debugging purposes, but are essential for the logic of dataset creation for the temporal masked modeling and multimodal fusion tasks, as well as for positive pair identification during the contrastive temporal alignment.

#### Signal Cleaning and Filtering

Each modality underwent modality-specific preprocessing and cleaning:

- **EEG:** A zero-phase second-order Butterworth bandpass filter was applied between 0.5 and 40 Hz using `scipy.signal.sosfiltfilt` [55]. This removed drift and high-frequency noise while maintaining the shape and timing of the EEG waveforms.
- **ECG:** Cleaned using `neurokit2.ecg_clean` [56], which applies a 0.5 Hz high-pass Butterworth filter, followed by powerline filtering for baseline correction and noise removal, in order to extract a clean ECG waveform that preserves the QRS complex morphology.
- **Respiration:** Processed using `neurokit2.rsp_clean` [56], which removes slow baseline drift and high-frequency noise by applying a second-order Butterworth filter between 0.5 and 3Hz.

To further mitigate the effect of transient noise, hard thresholds were empirically defined per channel to identify extreme amplitude values. Any sample beyond the specified range was flagged as an outlier and replaced using linear interpolation over valid neighboring points. This was done individually for each channel of each physiological signal.

### 3.5.3 Dataset Creation

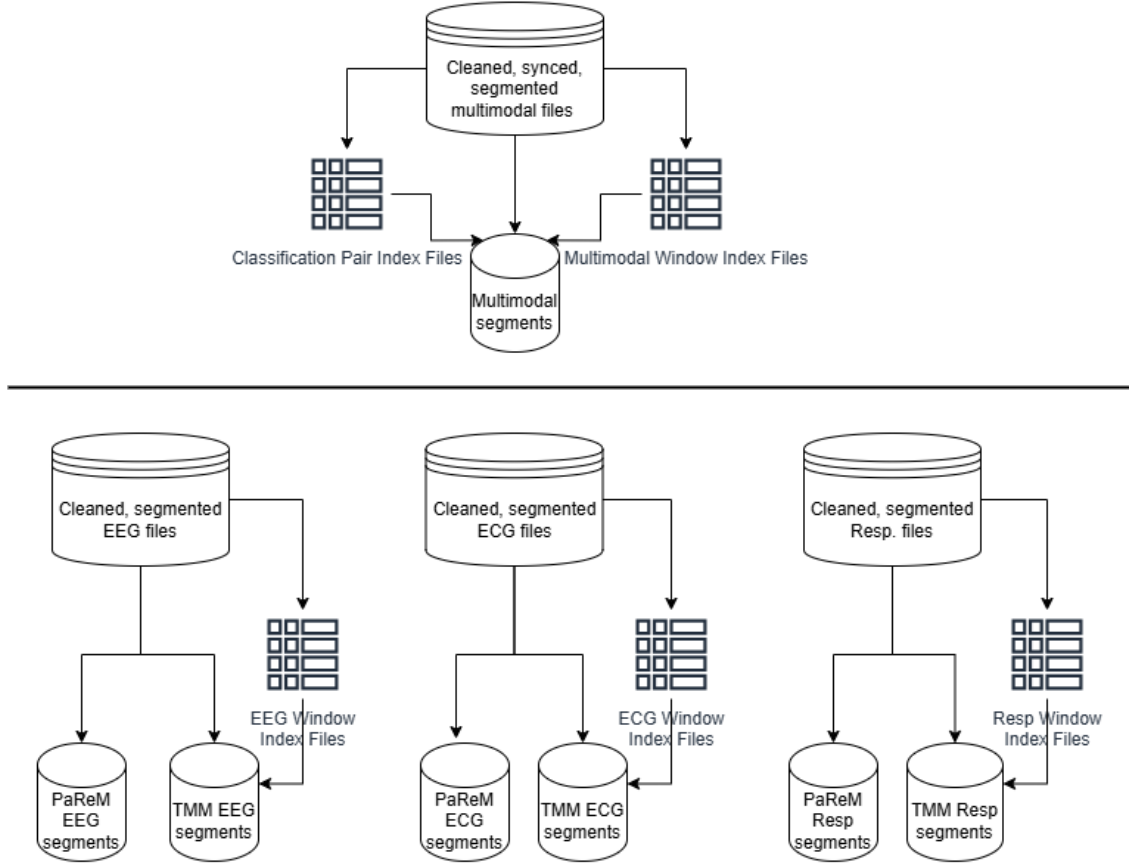


Figure 3.3: Overview of the dataset creation pipeline

The data cleaning and transformation tasks resulted in four .pk1 files for each participant: a multimodal file containing the 1-minute segments of cleaned and synchronized signals from each physiological signal along with the corresponding label; and a unimodal file with cleaned and segmented data for each modality. The next task was to combine these participant files and create a suitable dataset for each training stage satisfying the structural and semantic requirements. Finally, split the dataset of each stage into training, validation and test sets, and normalize the data according to the statistics computed on the training split. Figure 3.3 provides an overview of the dataset creation pipeline and the resulting data files for each training stage. Note that for each modality the PaReM segments are further divided into training, validation, seen test and unseen test splits, but these were not displayed separately on the figure to improve clarity.

To assess the generalization and memorization abilities of the proposed architecture, two separate test sets were established: one for Seen Participants and another for Unseen Participants. The Seen Participant Test Set is made up of segments from participants whose segments also appeared in the training or validation data, allowing us to assess performance in settings where individual-specific patterns may have been observed during training. By comparison, the Unseen Participant Test Set includes only segments from participants who were entirely excluded from all training and validation stages. This rigid separation offers a strong assessment of generalization to unfamiliar subjects—an important factor for practical implementation. To maintain uniformity during all training stages, a consistent group of held-out participants was utilized throughout, and their data was uniformly omitted from the creation of any modality-specific or multimodal datasets.

### **Dataset for MoCA-T**

To have any effect, the temporal alignment part of the MoCA-T pretraining task requires the presence of at least two segments from the same participant within a 10-minute radius from each other in each batch. However, due to the limited batch size of 32 imposed by memory constraints, the likelihood of such a pairing is unlikely in the case of entirely random batching. Therefore, a custom batching logic was created that guarantees temporally positive pairs for every segment in each batch, while still maintaining a certain level of randomness. This is achieved by constructing windows of four consecutive segments of a participant, then during training or inference, combining multiple randomly selected windows to form a batch, balancing diversity and structure. This ensures that each segment will have at least three temporally close segments in each batch, while the batches still contain segments from multiple participants, providing varied samples for the model. With three modality masked versions of each segment, this creates 66 positive pairs for a single window, therefore at least  $66 \cdot \left(\frac{\text{batch\_size}}{4}\right)$  positive pairs will be present in each batch. This design enables the use of the temporally enhanced contrastive learning objective.

These windows are created by processing each participant’s cleaned, synced, segmented multimodal file, and iterating over their data while saving disjunct lists of four segment indices, each list representing a window. These windows of indices are collected, and the ones not belonging to participants saved for the unseen participant test set are split into training, validation and test windows, with a ratio of 0.75, 0.15 and 0.1, respectively. The windowed indices of each split are saved in separate *Multimodal Window Index Files*.

The collected multimodal data segments are saved in a single file, after each channel of each physiological modality was Z-score normalized, using the mean and standard deviation of the training split’s data, selected by the saved windowed training indices.

Although each training stage requires distinct input structures and data handling, we can ensure that the underlying segment-level data used to construct the task-specific validation and test sets for the PaReM, MoCA-T and classification stages are consistently aligned. For this purpose, a flattened list of all the segment indices belonging to each data split created for MoCA-T is saved, as well as the validation and test segment indices grouped by participant.

#### **Dataset for Classification**

The classification task requires a dataset of paired multimodal segments where both segments in each pair belong to the same participant, but have differing drowsiness labels. A one-hot label is also needed for each pair signaling which of the two represent a drowsier state. The pair creation utilizes the collected and saved segment indices which represent the data splits defined during the dataset creation for the MoCA-T training stage. For each split, the corresponding data segments are grouped first by participant, then within each group, segments were further grouped by drowsiness label. Within each participant group pairs are defined by taking elements from differing drowsiness label groups. To ensure class balance, each valid label combination was used to construct two complementary samples - one for each possible order of the pair. Similarly to the MoCA-T dataset, instead of saving the actual paired data samples, only the indices of the pair get saved along with their one-hot class label. This design avoids redundantly saving the same data segment each time it appears in a pair, providing a memory-efficient and scalable solution. This strategy also enables the MoCA-T and classification stage to use the same base dataset, and create their task-specific samples and batches based on the simple index files.

Exhaustively generating all valid classification pairs resulted in approximately 212 000 training pairs, 9000 validation pairs, 3300 seen participant test pairs and 37 000 unseen participant test pairs. However, using the full training and validation set led to prohibitively long epoch duration, reaching over six hours per epoch. Due to time and GPU resource availability constraints, the training and validation sets were sampled to ensure feasible training times while preserving dataset balance and representativity. 5% of the training set was kept resulting in approximately 10 600 pairs, while 40% of the full val-

validation set was utilized, meaning around 3600 validation pairs. The evaluation was done on the full test sets to provide comprehensive assessment of model performance.

#### **Dataset for PaReM**

The Patch-Level Reconstruction with Momentum Targets training stage does not demand any special input structure or semantic condition, the input to the unimodal encoders simply consists of batches of independent unimodal 1-minute segments. Each segment is used exactly once in a single epoch, therefore a separate dataset was created for each modality and each split for straightforward data loading purposes. The saved indices grouped by participants for the validation and seen participant test splits defined during the MoCA-T dataset creation are utilized in order to ensure the alignment of the PaReM stage’s validation and test sets with the splits of the MoCA-T and classification stage. These indices are used to identify and collect the segments from the cleaned and segmented unimodal data files which correspond to the segments in the multimodal test and validation splits. The unimodal data files can contain additional segments compared to the multimodal dataset, since during the initial data preprocessing, for the unimodal dataset the valid unimodal data was kept even if the corresponding segment from another modality was deemed unusable. These auxiliary segments were included in the training data, in order to not disturb the validation and test split alignments. Before saving the created splits, the physiological signals are Z-score normalized based on the training split’s statistics.

#### **Dataset for TMM-SegRep**

The model used during the TMM-SegRep training stage operates over a window of consecutive 1-minute segments belonging to the same participant. Due to the limited dataset size, overlapping windows were created from the cleaned and segmented unimodal data using a sliding window approach with stride of one on each participant’s sorted segments. Following a similar logic as during the classification and MoCA-T dataset creation, instead of creating windows from the actual data segments, only the indices of the segments forming the windows were stored as lists, thereby redundantly storing each segment multiple times was avoided. The created windows belonging to participants outside of the unseen participant test set were divided into training, validation, dirty test and clean test splits. *Clean test* refers to windows which do not have any overlap with windows from

the training or validation split. This was achieved by iteratively selecting a random clean test window from a participant, then removing all overlapping windows from the available options. In case of the 16-minute windows applied in this study, this equaled to removing all windows if their start index was within a 15-minute radius of the chosen clean test window’s start index. Since this method leads to significant data loss, the number of clean test windows to choose per participant was defined based on the measurement length. By default, a single clean test window was selected from each participant’s data, but in case of sessions above 160 minutes, two windows were chosen, while for sessions which did not last at least 100 minutes, no windows were taken. These thresholds were defined empirically by analyzing the dataset composition and created measurement length histograms, with the goal of minimizing discarded data while still collecting a decent amount of clean test windows. The overlapping windows that were excluded from training and validation formed a separate *dirty test* set, enabling their partial utilization and creating a larger evaluation set. The unimodal data segments of each modality were saved into a single file, the splits were defined by the corresponding unimodal window index files. The signal of each segment was Z-score normalized with the training split’s mean and standard deviation.

While the validation and test splits across the PaReM, MoCA-T, and classification training stages were aligned to ensure consistent evaluation, the TMM-SegRep pretraining task required a different split strategy due to its input format and semantic requirements. As demonstrated in this subsection, TMM-SegRep processes windows of 16 consecutive 1-minute segment embeddings, and reserving a single clean window for test excludes any neighboring segments within a 15-minute radius from both sides of the chosen window. In the worst-case scenario (when the chosen window has at least 15 neighboring segments on both sides), this would exclude 46 minutes of data from TMM-SegRep training. Taking into account that the average measurement length was 140 minutes, even choosing just a single test window per participant would cause a significant reduction in training data, while reserving multiple windows would cause drastic data loss. Moreover, the drowsiness ratings were reported only every 5 to 15 minutes and exhibited limited variation within each participant session. On average, the range of self-ratings across a measurement was only 2.66, and the ratings typically followed a step-like progression — plateauing at a given level for several samples before gradually increasing and plateauing again. Considering all these characteristics, it is highly unlikely that a single 16-minute window chosen for testing would contain segments belonging to multiple drowsiness labels, thereby making them unsuitable for constructing binary classification pairs. As a result,



to avoid severely degrading the quantity of TMM-SegRep’s training data or the quality of the downstream tasks’ test data, the TMM-SegRep split was defined independently.

Although this setup allows for the possibility of overlap between the segment sequences used during TMM-SegRep pretraining and those in the classification seen participant test set, it does not lead to meaningful data leakage. The TMM-SegRep task is entirely unsupervised—it does not access any drowsiness labels—and it operates on segment sequences rather than isolated inputs. Even if a segment from the classification seen participant test set was included in a TMM-SegRep training window, it would not be optimized individually or with respect to any label information. Furthermore, the pretraining objective of TMM-SegRep differs fundamentally from the classification task, minimizing any potential for indirect leakage or memorization. Additionally, the sessions of participants chosen to form the unseen participant test set were excluded from the TMM-SegRep data as well, resulting in a perfectly aligned unseen participant test set across all training stages. This guarantees complete isolation of the unseen test set, eliminating any risk of data leakage and enabling a rigorous assessment of the models’ generalization capabilities. Therefore, while the data split definition is not strictly standard, it is well-justified given the constraints, and it preserves the integrity of the classification evaluation.

# Chapter 4

## Implementation

### 4.1 Architecture Design

The architecture is designed in a highly modular and configurable fashion that allows flexible experimentation. The components, such as unimodal encoders, temporal masked modeling component, and fusion encoder are defined in separate classes, making them easy to swap, extend, or fine-tune. For example, the fusion encoder can be replaced with an attention-based module by defining and instantiating a new class, and leaving the rest of the pipeline untouched. This facilitates ablation studies and future work, improves maintainability, and supports transfer learning, where previously trained components can be reused with ease.

#### 4.1.1 Unimodal Encoders

Each physiological modality (EEG, ECG, respiration) is processed by a dedicated unimodal encoder based on the EEG2Rep self-supervised learning framework[42]. These encoders are implemented using a modular Transformer-based architecture in PyTorch called `UnimodalEncoder`. The design ensures that modality-specific patterns are captured effectively, while maintaining consistency across modalities for later fusion. A common `Encoder` class is used for the implementation of the context and target encoder, while the predictor network is implemented by the `Predictor` class. To support PaReM pretraining, as well as future fine-tuning and inference, the `UnimodalEncoder` class includes a separate `forward` and `pretrain_forward` method.

## Input Embedding

The initial signal processing stages—including channel-wise and temporal convolutions, downsampling, and max pooling—are modularized into an `InputEmbedding` component. In our implementation, raw multichannel time-series signals are first reshaped via `x.unsqueeze(1)` to yield a tensor of shape  $(B, 1, C, T)$ , where  $B$  is the batch size,  $C$  the number of channels, and  $T$  the temporal length. We then apply a `Conv2d` with kernel size  $(C, 1)$  to transform the channel dimension into a  $d_{\text{emb}}$ -dimensional representation at each time point, followed by temporal filtering through successive `Conv2d` layers with kernel size  $(1, k)$ . Downsampling is achieved via strided convolution and max-pooling to produce a sequence of  $\lceil T/(s \cdot p) \rceil$  patches, where  $s$  and  $p$  denote convolutional stride and pooling factor, respectively.

Although mathematically equivalent to `Conv1d( $d_{\text{emb}} \times T \rightarrow d_{\text{emb}} \times T$ )`, the 2D formulation enables a unified and consistent API for both channel-wise and time-wise operations, simplifies use of padding modes (e.g., `padding='same'`, `padding_mode='reflect'`), and retains flexibility for future extensions (e.g., grouped or depthwise convolutions across both axes). This design yields clear separation between spatial transformation and temporal modeling, reduces tensor reshaping overhead, and ensures reproducible edge behavior.

Importantly, when  $C = 1$  (e.g., single-lead ECG or respiratory signals), the  $(C, 1)$  “channel-transform” convolution reduces trivially to a  $1 \times 1$  pointwise projection, thereby preserving the same unified embedding pipeline—avoiding special-case code paths—while still providing a learnable expansion to  $d_{\text{emb}}$  and all downstream temporal processing. This design abstracts the patch extraction logic into a reusable and configurable unit that outputs a sequence of fixed-size patch embeddings suitable for further processing by the encoder stack.

## Configuration

The unimodal encoder is highly configurable via a `config` dictionary, supporting:

- Variable patch number and embedding dimension,
- Variable input channel number and sequence length,
- Optional extra convolution layers, variable kernel size,
- Enable/disable flag for batch normalization,

- Variable transformer depth, attention heads, feedforward dimension,
- Optional FlashAttention support,
- Patch masking ratio, visible patch block number configuration.

A detailed list of all the configurable parameters and their default values can be found in the appendix.

### 4.1.2 Temporal Masked Modeling

The architecture for the TMM-SegRep pretraining task is implemented by the `TemporalMaskedModelingArchitecture` class. To follow a modular design, it receives a `UnimodalEncoder` instance through its constructor, and a separate `TMMTransformer` class is used to handle masked token prediction. That consists of a standard `nn.TransformerEncoder`, a light-weight MLP of two `Linear` layers with `ReLU()`, and a learnable mask token initialized with `nn.init.xavier_uniform_()`. The purpose of the `TMMTransformer` is to fine-tune the unimodal encoder's weights through the temporal masking loss, but its outputs are otherwise not used or forwarded to any other later module, therefore this class has a single forward function only.

#### Configuration

To facilitate experimentation, the TMM architecture supports the following tunable settings:

- Number of transformer layers, attention heads, feedforward dimension, dropout rate,
- Coefficients of loss components,
- Number of embeddings per input sequence,
- Masking ratio and number of mask versions to create per sample.

For a full specification of available configuration options and defaults, refer to the appendix.

### 4.1.3 Multimodal Fusion

The `DrowsiNet` class is the central multimodal encoder and fusion module in the architecture responsible for creating embeddings from multimodal signals by encoding and combining them. It serves as the backbone of `DrowsiRank`, however, due to its modular design, it can be easily used for any arbitrary downstream task which requires the creation of fused representations of physiological signals. Its forward method encapsulates the whole embedding process, which receives a dictionary of input tensors, one for each physiological modality (e.g., EEG, ECG, RSP). `DrowsiNet` is designed to operate on an arbitrary number of input modalities, each having a separate unimodal encoder model. These modality-specific encoders are provided as a dictionary and stored in a `ModuleDict`, which allows for dynamic composition of the model depending on the experiment setup. This design ensures that the architecture can easily be extended to support new modalities or replace current ones, without any structural change to the code. Moreover, the fusion encoder is configurable and easily swappable as well, it can be a simple feedforward network or any custom module such as a Transformer-based fusion block.

In addition, `DrowsiNet` implements the pipeline of MoCA-T pretraining, which logic is encapsulated in the `pretrain_forward` method. A learnable mask token is initialized for each modality to replace its entire corresponding embedding when applying modality masking. Since during pretraining VICReg loss is applied on the fused embeddings, which is a batch-level loss highly impacted by batch-size, the class includes convenience methods to provide true compatibility with gradient accumulation in the trainer in case of limited memory. These essentially split the logic of `pretrain_forward` into two stages: `get_embeddings` extracts and returns intermediate embeddings which can be cached in the trainer, and once enough latent representations have been accumulated, `compute_losses_from_embeddings` applies pretraining losses on these cached latent representations. To facilitate ablation studies and experimentation, besides the `temporal_contrastive_loss` method which supports our proposed logic of considering temporally close segments as positive pairs, an `exact_match_contrastive_loss` method is also available. This only counts differently masked versions of the exact same input sample as positive, and calculates MSE only between the fused embeddings of those.

## Configuration

The behavior and structure of the `DrowsiNet` module are governed by a set of configurable parameters, including:

- Number and type of modalities,
- Coefficients of the loss components,
- Type of contrastive loss function to use during pretraining,
- Temporal radius within which positive pairs are defined.

A full parameter reference, including default values, is included in the appendix for reproducibility and clarity.

### 4.1.4 Classification

The `DrowsiRank` model is designed as a modular architecture for pairwise comparison of global drowsiness embeddings. The class is implemented to receive any embedding-generating model as a pluggable backbone (e.g., `DrowsiNet`). This modular design allows the same ranking head to be reused with different underlying feature extractors.

To ensure reusability across diverse experimental scenarios, the model supports two *feature combination strategies* to compare two embeddings  $\mathbf{z}_i$  and  $\mathbf{z}_j$ :

- **Concatenation** ("concat"): the embeddings are concatenated as  $[\mathbf{z}_i; \mathbf{z}_j]$ , producing a vector of size  $2d$ .
- **Difference** ("diff"): a directional comparison is performed via vector subtraction  $\mathbf{z}_i - \mathbf{z}_j$ , resulting in a vector of size  $d$ .

This behavior is configurable using the `combination` argument in the constructor.

The comparison vector is passed to a two-layer feedforward classifier with an optional dropout layer. The hidden layer size and dropout probability are configurable via the `hidden_dim` and `dropout_rate` parameters. The default output is a binary classification score indicating which of the two inputs is predicted to have *higher* drowsiness.

## 4.2 Dataset Implementation

### 4.2.1 Custom Dataset Structures

To support different stages of model training and evaluation, several task-specific datasets were constructed.

- **UnimodalDataset:** Used for pretraining the unimodal encoders with the PaReM pretraining task. Each sample contains a single 1-minute segment of EEG, ECG, or RSP data, along with participant and segment metadata.
- **IndexedWindowDataset:** Designed for TMM-SegRep. Loading full sequences would mean redundantly storing each 1-minute segment every time it appears in a windowed sequence, leading to large and unnecessary memory consumption. Instead, the dataset receives a single pool of unique 1-minute segments and pre-computed lists of indices representing each window, allowing it to return temporally contiguous sequences of unimodal embeddings (e.g.,  $16 \times 1$ -minute segments) in a memory-efficient way.
- **MultimodalDataset with WindowBatchSampler:** Used for MoCA-T pretraining. Each sample includes all three modalities, the index of the segment and the participant ID to allow positive pair identification during the contrastive pretraining phase. The dataset is paired with a custom WindowBatchSampler to build batches from multiple index-based mini-batches of temporally close segments to ensure the presence of positive pairs in each batch while still maintaining variability.
- **PairwiseDataset:** Used for the final classification task, where the model must decide which of two samples corresponds to a higher drowsiness level. Each sample is composed of two index lookups into the dataset, as specified by a separate file of pairwise combinations. Each returned entry contains a pair of aligned EEG, ECG, and respiration segments belonging to the same participant, along with a one-hot supervision label indicating relative drowsiness, where the value 1 represents the sample with the higher drowsiness rating.

### 4.2.2 Storage-Efficient Data Handling and Loading Strategy

To enable memory-efficient training on large multimodal sequences, all datasets are backed by the `datasets` library from Hugging Face [57]. Raw `.pkl` data files are initially loaded using `load_dataset` and preprocessed with a custom `.map()` function. The processed datasets are then saved with `save_to_disk`, enabling rapid reuse in subsequent runs and no need for repeated preprocessing. Each unimodal and multimodal dataset and fold combination is saved to a unique path.

Each dataset is wrapped in a custom PyTorch Dataset subclass [58], which receives the path to the disk-saved data, and loads it with `load_from_disk`. To reduce memory usage, only the relevant columns for the current training stage are kept. These classes are further combined with standard `DataLoader` wrappers that use consistent flags (`num_workers=4`, `pin_memory=True`, `persistent_workers=True`) for efficient parallel loading. Where applicable, to support reproducibility, shuffling is made deterministic by passing a seeded `torch.Generator`, and advanced sampling strategies are supported via custom `Sampler` implementations.

All custom dataset classes return samples as PyTorch tensors. This design avoids on-the-fly conversions in the training loop, streamlining GPU compatibility and reducing runtime overhead.

## 4.3 Training infrastructure

### 4.3.1 Hardware and Training Environment

The training was conducted on the private Central Processing Unit (CPU) and Graphics Processing Unit (GPU) resources of Robert Bosch Ltd./Bosch Group, Hungary. For the majority of training, including the pretraining tasks of unimodal encoders, temporal masked modeling and modality masking with temporal alignment, a machine with two NVIDIA GeForce GTX 1080 Ti GPUs with 11GB of Video Random-Access Memory (VRAM) were utilized. This posed a bottleneck concerning the size of the models, the maximum batch size, training speed and available PyTorch implementations. For the classification training, a machine equipped with an NVIDIA A100 GPU with 40GB VRAM was used.



### Environment

To ensure consistent execution across both systems, a Docker container was used, created from the custom image. This setup helped avoid inconsistencies due to system-specific configurations and provided a stable and reproducible software environment throughout development. The parameters of the core environment:

- PyTorch version: 2.6.0 [58]
- CUDA version (PyTorch): 11.8
- cuDNN version: 90100

### Compatibility

Since NVIDIA GeForce GTX 1080 Ti GPUs have a compute capability of 6.1, advanced CUDA features such as dynamic compilation via `torch.compile()` and optimized attention kernels like FlashAttention with `torch.nn.functional.scaled_dot_product_attention` were not supported on that machine. As a result, these optimizations were disabled when running on the 1080 Ti, and training was limited to compatible fallback paths, making the process significantly slower and more memory-consuming.

### 4.3.2 Gradient Accumulation and Batch Management

Since all pretraining phases depend on batch-level loss components as well, such as the feature-wise variance in the VICReg loss, it was important to have a representative batch size. To support a larger effective batch size than what the GPU memory constraints would normally allow, either gradient accumulation or embedding accumulation was used during training. Multiple forward passes were performed on mini-batches and their corresponding embeddings were aggregated before computing the loss and performing a single backward pass once the number of aggregated mini-batches reached a specified accumulation threshold. This approach allows the model to simulate larger batch sizes while keeping GPU memory usage manageable.

### 4.3.3 Precision Handling

To improve training efficiency and allow larger batch sizes while taking into account memory constraints, automatic mixed precision (AMP) was used throughout each training process. AMP enables a combination of float16 and float32 operations, reducing memory usage and increasing the speed of computations. During the forward pass, mixed precision was enabled using `torch.amp.autocast`, which casts operations to float16 where safe, while keeping accuracy-critical operations in float32.

Loss calculation and backpropagation were performed with the help of `torch.amp.GradScaler`. The scaler dynamically adjusts the scaling factor of the computed loss to prevent underflow during gradient computation due to the low precision. First, the loss is scaled and backpropagated with the scaler's `scale(loss).backward()` method, then the gradients are unscaled and used for optimizer updates with the `step(optimizer)` method, followed by a scaler update step.

### 4.3.4 Training Loop and Logging Infrastructure

For the sake of maintainability and reproducibility across experiments, each pretraining stage was handled by a custom `Trainer` class. The training loop handles all necessary stages of model optimization, including forward pass, backward pass, gradient scaling, validation, checkpointing, and early stopping. All stages support mixed precision training via PyTorch's `autocast` and `GradScaler`, and optionally apply gradient accumulation when enabled through configuration.

Logging was performed using `torch.utils.tensorboard` through a `SummaryWriter` attached to each run. The trainer automatically created run-specific directories for logs and checkpoints, saving all configuration files to enable reproducibility. For each epoch several metrics were recorded, such as training loss components, gradient norms, feature statistics, and learning rates. In addition, histograms of weight and gradient distributions were periodically logged for deeper inspection.

### 4.3.5 Reproducibility

To ensure reproducible experiments and consistent results across runs, all random number generators were seeded using a configurable global seed value (42 by default). Additionally, PyTorch was set to deterministic execution mode with `torch.backends.cudnn.deterministic = True` and benchmarking got disabled with `torch.backends.cudnn.benchmark = False`, as shown in Code 4.1.

```
1 def set_seed(seed):
2     random.seed(seed)
3     np.random.seed(seed)
4     torch.manual_seed(seed)
5     torch.cuda.manual_seed(seed)
6     torch.cuda.manual_seed_all(seed)
7     torch.backends.cudnn.deterministic = True
8     torch.backends.cudnn.benchmark = False
```

Code 4.1: Seed initialization for full reproducibility

The seed of each PyTorch data sampler or batch sampler was fixed to make data loading deterministic as well. Model checkpoints, logs, and the exact model configuration were stored for each run in separate directories named with a timestamp or a custom run name passed by the user. All training scripts follow a consistent structure, making the full recovery and re-run of any previous experiment possible.

### 4.3.6 Checkpointing and Weight Loading

To support flexible model initialization and ensure recoverability, all training stages included thorough checkpointing and weight-loading. During training, model checkpoints were automatically saved at regular intervals and when the best validation loss was achieved. These checkpoints included all necessary details for the training to be continued from there, such as the model weights, optimizer state, and current epoch number.

Weight loading was used extensively to initialize different components with pretrained parameters using PyTorch's `torch.load` function with device-specific mapping. To facilitate this, checkpoints were organized into different folders for each training phase, and only relevant components were extracted and loaded. A custom initialization logic excluded unnecessary components from fine-tuning by disabling their gradients.

### 4.3.7 Optimization and Parameter Freezing Strategy

All models were trained using PyTorch’s Adam optimizer. Later training stages — such as TMM-SegRep, MoCA-T, classification — build on pretrained components. In order to preserve previously learned features and allow stable adaptation of newly introduced layers (e.g., classifier or TMM heads), initially, the pretrained layers’ parameters were frozen. After a configurable number of epochs, these earlier layers were also unfrozen to allow for their fine-tuning. The optimizer was reinitialized, optionally with separate learning rates for newly unfrozen and previously trainable components, carefully selecting from the earlier architectures only the parameters useful in the current training stage (e.g. logically the predictor and mask token parameters in the unimodal encoder were left out of optimization in later training stages). This unfreezing strategy improved stability and allowed precise control over adaptation rates of different modules.

The learning rate was scheduled using a cosine annealing policy implemented via PyTorch’s `CosineAnnealingLR` scheduler. The scheduler was reinitialized after the unfreezing event to align the decay schedule with the number of remaining training epochs.

# Chapter 5

## Experiments and Results

### 5.1 Pretraining experiments

Each of the three pretraining stages— PaReM, TMM-SegRep, and MoCA-T —was trained separately. While the model architecture and optimization setup were largely consistent across experiments, key hyperparameters of each training stage were iteratively adjusted based on the monitored training dynamics and validation losses. Conducting an exhaustive search was not feasible as part of this thesis due to time and GPU resource constraints, and the large number of hyperparameters. Therefore, parameter values were selected through empirical testing and domain-driven intuition, aiming to improve stability and generalization at each stage.

The hyperparameters not mentioned in the below sections used the default values in each stage, as defined in Appendix A. These default values were chosen following common practices in the literature, e.g. the size of the intermediate feedforward layer in the Transformer blocks of the unimodal encoders was set to four times the embedding dimension. An effective batch size of 32 was kept for each training stage. In total, 195 trainings were conducted, with an overall 361 hours of training time.

Throughout all pretraining stages, special attention was paid to the variance component of the VICReg loss, which plays a critical role in avoiding representation collapse. Without this component, the model could converge to trivial solutions where the masking-based reconstruction or alignment losses are minimized by simply producing nearly constant or identical embeddings for all inputs. To prevent this issue, the variance loss was explicitly tracked during training, and in case of model variants with on-par performance, the one with lower variance loss was preferred.

### 5.1.1 PaReM pretraining

Each unimodal encoder was trained separately on the NVIDIA GeForce GTX 1080 Ti GPUs, using the PaReM objective. The number of epochs was set to 100, with early stopping after 15 epochs of no validation loss improvement. Since these encoders form the foundation of all later stages and they had a relatively short training time between 2-4 hours, a series of targeted hyperparameter searches were conducted:

- **Embedding Size:** Models were trained with the unimodal embedding size set to 16, 32, 64, 128, 256. While smaller dimensionality accelerated training, it caused inconsistencies and significantly higher loss across modalities, signaling that the models struggled to compress information adequately into smaller representations. Embedding size of 128 and 256 were the best performing ones, therefore these were carried over to later training stages as well.
- **Transformer Layer Configuration:** Several combinations of the number of layers in the context (and target) encoder and the predictor were tested. For EEG, a deeper configuration of 4 encoder and 4 predictor layers achieved best validation loss values. For ECG, 3 encoder and 2 predictor layers was chosen, while the architecture for respiratory signal was kept lightweight with only 2 encoder and 2 predictor layers.
- **Patch Granularity:** Two configurations were evaluated for the number of input patches: 480 and 960. The final setting used 960 patches, providing finer temporal resolution for downstream tasks.
- **Visible Blocks and Mask Ratio:** A higher masking ratio of 0.5 (vs. 0.25) was adopted to encourage stronger reconstruction learning, while the semantic subsequence preserving method achieved best performance with 10 visible blocks (out of the tried 3,5,10,15).
- **Learning Rate:** Learning rates of 0.01, 0.001, and 0.0001 were explored. A learning rate of 0.001 yielded the best balance of stable training and good convergence across modalities.

### 5.1.2 TMM-SegRep pretraining

Following the unimodal pretraining, each encoder was further trained using the TMM-SegRep objective, which aimed to model longer-range temporal dependencies by learning to reconstruct hidden representations from multiple consecutive 1-minute embeddings. The encoders were initialized with the pretrained weights obtained from the PaReM task. To avoid disrupting the learned features, the encoder layers were frozen for the first 10 epochs, allowing the TMM head to stabilize first. The trainings were run for 100 epochs on the NVIDIA GeForce GTX 1080 Ti GPUs, with early stopping patience of 15. In this stage training time significantly increased compared to the PaReM trainings, the average training time surpassing 10 hours.

To improve performance, several hyperparameters were explored:

- **Learning Rate:** Two learning rates were evaluated: 0.001 and 0.0001. While the lower rate offered slightly more stable gradients, training with a learning rate of 0.001 led to faster convergence without sacrificing performance, and was therefore selected for the final configuration.
- **Masking Ratio:** The ratio of masked segments within each sequence sample was chosen to be 0.5, since it outperformed the 0.25 variants by encouraging stronger temporal reasoning.
- **Mask Number:** Increasing the number of masked versions created from each training sample from 2 to 4 showed better validation results although it caused longer training times.
- **TMM Head Layer Depth:** Interestingly, a shallower architecture of a 1-layer transformer in the TMM module performed better in this setting than the 2-layer variant. Since deeper models typically require more data to generalize effectively, this behavior might be a consequence of the relatively limited training dataset (approximately 3,000 samples). Additionally, since the inputs to the TMM module are already high-level segment representations, the marginal benefit of additional model depth may be reduced.

### 5.1.3 MoCA-T pretraining

For the Modality-Contrastive Alignment with Temporal Positives (MoCA-T) pretraining stage, the encoders were either trained from scratch, or they were initialized with the pretrained weights obtained from the PaReM task or TMM-SegRep task. To prevent losing the acquired knowledge of the pretrained encoders, their layers were trained only after the initial 15 epochs, enabling the fusion encoder to adapt to the learned representations first. The trainings were run for 100 epochs on the NVIDIA GeForce GTX 1080 Ti GPUs, with early stopping patience of 15. Training time varied between 3 and 6 hours.

Initial experiments showcased that using the same learning rate for the fusion encoder and the unfrozen backbone led to unstable gradients, increasing and fluctuating validation loss, and hindered convergence. To address this, a separate and lower learning rate of  $10^{-4}$  was applied specifically to the unimodal encoders (the backbone), while the rest of the model used the default rate of  $10^{-3}$ . This modification consistently stabilized training and allowed the proper fine-tuning of encoders.

After some preliminary experimentation with the fusion encoder’s structure, the lightweight feedforward neural network version was chosen over the Transformer-using variant. This consisted of 3 Linear layers, with input and output dimensions of  $(\text{embedding\_dim} \times \text{modality\_num}, 2 \times \text{embedding\_dim})$ ,  $(2 \times \text{embedding\_dim}, 2 \times \text{embedding\_dim})$ , and  $(2 \times \text{embedding\_dim}, \text{embedding\_dim})$  respectively, each followed by a ReLU activation, and a final layer normalization was applied. Another variant was tried with a slightly narrower middle section of  $(\text{embedding\_dim}, \text{embedding\_dim})$  instead of  $(2 \times \text{embedding\_dim}, \text{embedding\_dim})$ , but this presented worse results.

Even in this stage, the effect of robust representation learning through multiple chained pretraining tasks was observable. Pretrained models had a lower training and validation loss from the beginning, and they achieved an overall lower best validation loss than the variants trained from scratch.



## 5.2 Classification experiments

### 5.2.1 Model Variants

To the best of our knowledge, no publicly available dataset provides parallel EEG, ECG, and respiratory signals collected in a driving setting, nor are there existing benchmarks for this specific multimodal configuration and task. Therefore to systematically evaluate the contribution of the proposed pretraining objectives and temporal enhancements, five model variants were defined. These variants shared an identical hyperparameter setup, and differed only in their weight initialization strategies—depending on which pretraining tasks were applied—and in whether the unimodal encoders contained a context encoder or only convolutional layers. This design allowed for a controlled comparison of the effects of the pretraining tasks and the impact of additional temporal modeling within the unimodal encoders. The created variants are the following:

- **Full Pretraining:** The classification was preceded by all three pretraining phases—PaReM, SegRep-TMM, and MoCA-T-, and the unimodal encoders included a context encoder.
- **No TMM-SegRep:** Identical to the fully pretrained model but without the segment-level temporal masked modeling task. This variant isolates the effect of TMM-SegRep on final performance.
- **MoCA-T Pretraining:** Pretrained only with MoCA-T task, this variant evaluates the importance of intra-segment temporal modeling within each modality. Uses only convolutional layers in the unimodal encoders, removing the context encoder.
- **MoCA Pretraining:** Same as the previous variant, but pretrained using only modality masking without temporal alignment (by computing `exact_match_contrastive_loss`), assessing the latter’s added value.
- **No Pretraining:** The model was trained entirely from scratch without any pretraining tasks. The unimodal encoders used convolutional layers only, without a context encoder.

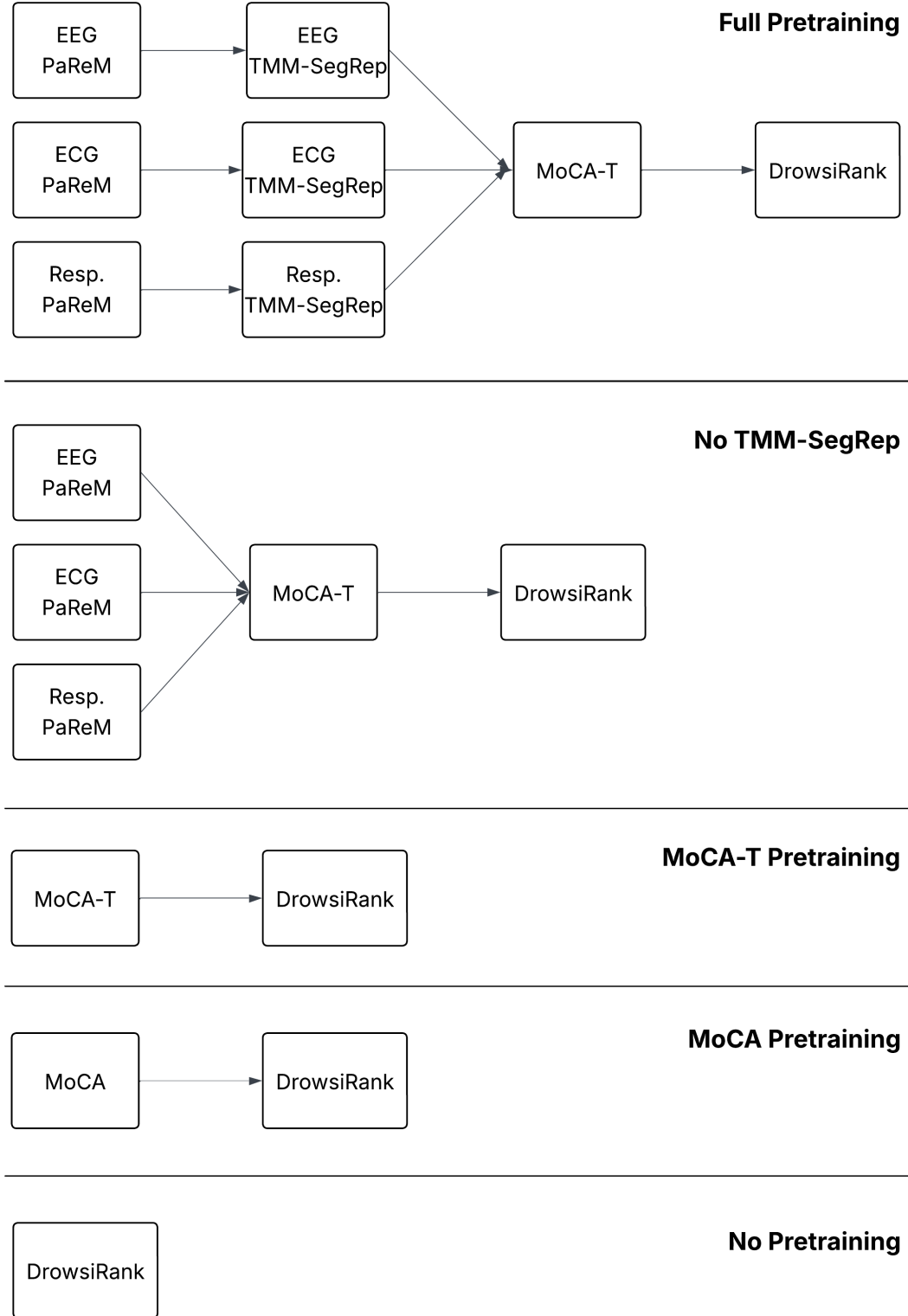


Figure 5.1: Flowchart of chained pretraining stages for each model variant

The chained sequence of pretraining stages for each model variant is illustrated in Figure 5.1. Later stages build upon the weights of unimodal encoders and, when applicable, the fusion encoder, obtained from the preceding pretraining stage.

### 5.2.2 Hyperparameters and Optimization Strategy

For the sake of fair comparison, all model variants were trained using the same classification setup. The final hyperparameters were selected through manual tuning based on validation performance over 76 trainings taking 355 hours. Initially, the models showed signs of rapid overfitting, therefore dropout regularization was introduced in the classifier head. Multiple values were tested, and subsequently, a stronger weight decay was applied to further improve generalization. Several learning rates were explored (0.01, 0.001, 0.0001) but a learning rate of  $1 \cdot 10^{-3}$  yielded the best overall performance. For model variants including pretraining stages, the pretrained weights were frozen for the first few epochs to allow the classifier head to stabilize. 5 frozen epochs provided a good balance between stability and adaptation. After analyzing the performance of models with both 128- and 256-dimensional embeddings, 128 got chosen since it consistently achieved lower validation loss values over all training stages, and required shorter training time. Therefore, the final configuration applied across all variants used a learning rate of  $1 \cdot 10^{-3}$ , weight decay of  $1 \cdot 10^{-2}$ , a dropout rate of 0.4, embedding dimension of 128 and an effective batch size of 32 pairs. The models were trained on the NVIDIA A100 GPU with model compilation for 100 epochs, with early stopping after 10 non-improving epochs. Most trainings took between three and five hours until model convergence.

### 5.2.3 Evaluation Protocol

To evaluate generalization and robustness, two distinct test sets were utilized: The **Seen Participant (SP) Test Set** contains segments from participants whose other segments appeared during training or validation, while the **Unseen Participant (UP) Test Set** contains segments from entirely held-out participants, whose segments were not included in the training or validation data.

For a more comprehensive assessment, three distinct *evaluation splits* were created. In each split, three participants were completely held out for the Unseen Participant Test Set, while the remaining participants were used to construct the training, validation, and Seen Participant Test Set splits. The held-out participants differed across the three evaluation splits to provide diversity. Each model variant was trained and evaluated on all three evaluation splits using the same hyperparameter configuration on each.

### 5.2.4 Outcomes

| ID    | Split | PaReM | TMM-SegRep | MoCA | Temp | UP Acc        | SP Acc        |
|-------|-------|-------|------------|------|------|---------------|---------------|
| exp1  | 1     | ✓     | ✓          | ✓    | ✓    | <b>57.2%</b>  | 74.58%        |
| exp2  | 1     | ✓     | ✗          | ✓    | ✓    | 55.39%        | 75.24%        |
| exp3  | 1     | ✗     | ✗          | ✓    | ✓    | 53.47%        | 80.02%        |
| exp4  | 1     | ✗     | ✗          | ✓    | ✗    | 54.51%        | 80.11%        |
| exp5  | 1     | ✗     | ✗          | ✗    | ✗    | 37.36%        | <b>84.46%</b> |
| exp6  | 2     | ✓     | ✓          | ✓    | ✓    | 64.13%        | 77.09%        |
| exp7  | 2     | ✓     | ✗          | ✓    | ✓    | <b>64.79%</b> | 73.26%        |
| exp8  | 2     | ✗     | ✗          | ✓    | ✓    | 55.38%        | 70.18%        |
| exp9  | 2     | ✗     | ✗          | ✓    | ✗    | 54.75%        | 67.53%        |
| exp10 | 2     | ✗     | ✗          | ✗    | ✗    | 59.16%        | <b>79.59%</b> |
| exp11 | 3     | ✓     | ✓          | ✓    | ✓    | <b>60.65%</b> | <b>82.19%</b> |
| exp12 | 3     | ✓     | ✗          | ✓    | ✓    | 55.79%        | 80.32%        |
| exp13 | 3     | ✗     | ✗          | ✓    | ✓    | 56.59%        | 68.42%        |
| exp14 | 3     | ✗     | ✗          | ✓    | ✗    | 50.46%        | 74.5%         |
| exp15 | 3     | ✗     | ✗          | ✗    | ✗    | 48.74%        | 81.60%        |

Table 5.1: Classification performance across three evaluation splits for various pretraining configurations on unseen participants (UP Acc) and seen participants (SP Acc). Each row represents a single training of a model variant. Bold values indicate the best accuracy within each fold for both test sets.

Table 5.1 presents the classification performance of the proposed DrowsiRank model variants across three evaluation splits for both test sets, while Table 5.2 shows the averaged accuracy scores with their 95% confidence interval.

#### Results for Unseen Participants

In two out of three splits (Split 1 and Split 3), the fully pretrained model variant achieved the highest accuracy on the Unseen Participant Test Set, and even in Split 2 its accuracy of 64.13% is only marginally lower than the best score of 64.79%, placing it among the top-2 in all splits. The fully pretrained model scored the best overall unseen participant accuracy with 60.66%, meaning a 12.24% increase compared to the model with no pretraining, and an improvement of 7.42% over the MoCA pretrained baseline variant without any temporal extensions.

The more pretraining stages added, the better the overall unseen participant accuracy becomes. Even when only MoCA is used, the UP accuracy shows an improvement of 4.82% compared to the variant without any pretraining. Extending MoCA with temporal

alignment caused a 1.91% accuracy increase. We suspect that with a bigger batch-size where the probability of encountering temporally close segments from the same participant is higher, this effect would be more significant. Introducing a PaReM-pretrained context encoder yielded a 3.51% gain in performance, suggesting that incorporating additional temporal encoding into unimodal embedding creation encourages stronger global representations. When TMM-SegRep was left out, accuracy dropped from 60.66% to 58.66%, which highlights the benefit of capturing temporal dependencies across 1-minute segments. These results indicate that the temporal enhancements indeed improve the model’s robustness. By learning multi-level temporal features and connections, the model creates better representations of signals, and can generalize better to new participants.

| PaReM | TMM-SegRep | MoCA | Temp | UP Acc                    | SP Acc                    |
|-------|------------|------|------|---------------------------|---------------------------|
| ✓     | ✓          | ✓    | ✓    | <b>60.66%</b> $\pm$ 3.92% | 77.95% $\pm$ 4.39%        |
| ✓     | ✗          | ✓    | ✓    | 58.66% $\pm$ 6.01%        | 76.27% $\pm$ 4.12%        |
| ✗     | ✗          | ✓    | ✓    | 55.15% $\pm$ 1.78%        | 72.87% $\pm$ 7.07%        |
| ✗     | ✗          | ✓    | ✗    | 53.24% $\pm$ 2.73%        | 74.05% $\pm$ 7.13%        |
| ✗     | ✗          | ✗    | ✗    | 48.42% $\pm$ 12.34%       | <b>81.88%</b> $\pm$ 2.77% |

Table 5.2: Averaged classification performance on unseen (UP) and seen participants (SP) across evaluation splits and model variants, reported as mean  $\pm$  95% confidence interval. Our pipeline shows improved generalization.

### Results for Seen Participants

The highest seen participant accuracies show different tendencies, and are not always aligned with the best unseen participant results. For example, in Split 1, the model without any pretraining (exp5) reached the best test accuracy (84.46%), while its performance on unseen subjects significantly lags behind (37.36%), scoring last place in that split. This pattern is also reflected in the model variant’s overall results, it achieved the highest seen participant accuracy (81.88%), however, the lowest unseen participant accuracy (48.42%). These values suggest that models lacking structured pretraining may overfit to noise or misleading correlations, and can not generalize to new participants, achieving even worse results than random guessing. In Split 3, the fully pretrained model achieved the best score on both test sets. Notably, the overall SP accuracies suggest that including a PaReM and TMM-SegRep pretrained context encoder, thereby urging the model to capture more temporal patterns, results in superior performance for familiar participants as well.

### Performance Trade-off

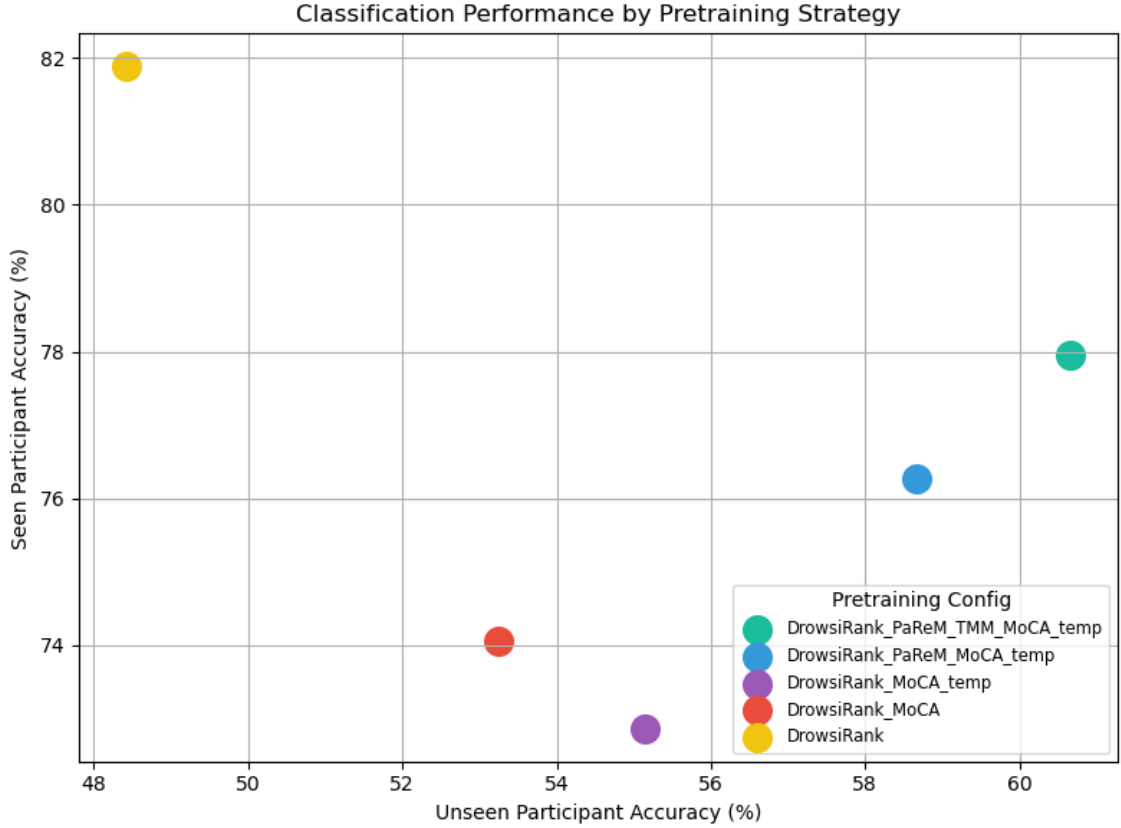


Figure 5.2: Relationship between Unseen Participant and Seen Participant Test Accuracy Across Pretraining Configurations. Each point represents a unique combination of pretraining strategies.

Figure 5.2 visualizes the relationship between the accuracies on the two test sets, highlighting the trade-off between raw performance on seen data and generalization to unseen participants. Across all splits, the difference between SP and UP accuracy can be quite large (up to 47.1% in the case of exp5), reinforcing the importance of evaluating models on held-out individuals to properly assess generalization performance. SP accuracy is generally high (above 72%) across all variants, indicating that the task is easier when the model has seen similar data during training. Due to the limited participant number and diversity of the dataset, this observed performance gap was anticipated. However, the most balanced configuration appears to be the fully pretrained variant with the best unseen participant accuracy of 60.66% and the second best seen participant accuracy of 77.98. These results are encouraging, and with the inclusion of more participants and larger dataset, we expect an improved generalization and overall performance, especially for pretrained models.

### Variance and Stability Across Folds

The proposed full model shows relatively narrow confidence intervals across splits:  $\pm 3.92\%$  for the accuracy on the Unseen Participant Test Set and  $\pm 4.39\%$  on the Seen Participant Test Set’s accuracy. The MoCA-T pretrained version showed the most stable unseen participant performance. In contrast, the model variant without pretraining has the widest confidence interval ( $\pm 12.34\%$ ) on unseen test accuracy, indicating sensitivity to the choice of held-out participants. Although in Split 2 it achieves 59.16%, even outperforming the MoCA and MoCA-T pretrained versions, its performance on other splits suggests that this was not due to true generalization. On the other hand, when it comes to seen participant accuracy, it showed the least fluctuation and the highest score ( $81.88\% \pm 2.77\%$ ). This contrast further supports the hypothesis that the model variant may rely more heavily on participant-specific patterns, failing to capture features that generalize well across individuals.

### Conclusion

While the best classification accuracy on the Unseen Participant Test Set was around 60%, this outcome is expected given the limited number of participants in the current dataset. However, the observed performance improvements for model variants utilizing additional temporal modeling indicate a promising trend towards better generalization. With additional data collection and training, we expect the performance advantage of temporally boosted models to increase compared to the baseline variants with no context encoder and no pretraining or modality masked pretraining only.

Overall, the results support our hypothesis that incorporating multi-level temporal modeling enhances the model’s ability to yield more stable representations for downstream drowsiness classification. The full pretraining pipeline, including PaReM, TMM-SegRep and MoCA-T, performed the best on the Unseen Participant Test Set. While on familiar participants the model variant without pretraining achieved the best results, it failed to generalize as effectively, achieving the worst accuracy score on the Unseen Participant Test Set. This emphasizes the advantage of combining all pretraining tasks for robust multimodal understanding.

# Chapter 6

## Future Work

**Downstream Tasks and Latent Space Exploration** The pretrained unimodal and fusion encoders and the resulting multimodal latent space offer exciting further research opportunities and potential for additional downstream tasks beyond pairwise classification. In-depth analysis of the latent space—such as clustering, temporal trajectory visualization, comparison with self-reported drowsiness ratings, and their changes throughout different pretraining stages—could provide insights into how physiological states are encoded and differentiated. Moreover, while this work concentrated on drowsiness detection in a driving context, the proposed model architecture could also be applied and tested in other domains, such as industrial safety.

**Data Collection and Demographic Diversity** The current dataset introduced in Section 3.5 comes with multiple limitations, such as the small number of participants, lack of demographic diversity (all-male participants belonging to a limited age range), and the use of a simulated environment. Future studies should focus on collecting additional and more demographically representative data to improve the generalizability and fit of the model for real-world scenarios.

**Hyperparameter Optimization** Although this work included manual tuning of key hyperparameters as described in Section 5.1, due to the large hyperparameter number across multiple stages and time constraint, a full search was not conducted. Future work could explore the effect of different hyperparameter configurations on the model performance, and perform more systematic optimization e.g. with grid search.



# Chapter 7

## Conclusion

This thesis explored multimodal deep learning approaches for driver drowsiness detection, pointing out the relevance of pretraining strategies, temporal modeling, and robust evaluation protocols. The main aim was to develop a model with improved generalization capabilities to unseen participants by creating robust multimodal representations through sequential pretraining stages and introducing multi-level temporal awareness.

A modular and extensible framework was introduced that allows flexible integration of multiple signal modalities and various self-supervised objectives. The proposed pipeline described in Section 3.4 consisted of a sequence of chained pretraining stages: Patch-Level Reconstruction with Momentum Targets (PaReM), Temporal Masked Modeling over Segment-Level Representations (TMM-SegRep), and Modality-Contrastive Alignment with Temporal Positives (MoCA-T). Each pretraining stage built upon the weights learned in the preceding stage, forming a cohesive and hierarchical representation learning process. Finally, a classifier was trained to recognize pair-wise increased drowsiness states using the pretrained encoders as a feature extraction backbone.

The evaluation was carried out over three different train-test splits, each holding out a distinct subset of participants from the training and validation data to assess generalization performance. Due to the uniqueness of the domain and the scarcity of public multimodal physiological datasets, to the best of our knowledge, there are no available benchmarks for relative driver drowsiness ranking based on the combination of EEG, ECG and respiratory signals. Therefore, to assess the added value of the proposed temporal enhancements, extensive classification experiments were conducted to compare five model variants, described in Section 5.2, which differed only in their combination of used pretraining stages and the presence of intra-segment context encoders.

The results showcased in Table 5.2 and Table 5.1 demonstrated that models incorporating temporal modeling through all three pretraining stages achieved the best performance on the Unseen Participant Test Set, confirming the effectiveness of progressive representation learning in inter-person generalization. The fully pretrained model equipped with an intra-segment context encoder reached an average accuracy of 60.66% on unseen participants, outperforming all other variants and resulting in an improvement of 12.24% compared to the baseline model without any pretraining or additional temporal modeling. It also retained performance on seen participants degrading less than 4% in classification accuracy with a score of 77.95%, proving to be the model variant which best balances inter-personal generalization and intra-personal memorization. Our analysis shows that without any pretraining models tend to overfit on the physiological signals of seen participants while they exhibit the lowest accuracy and the widest confidence interval on unseen data, suggesting a lack of robustness to new participants. Additionally, our ablation experiments show that each added stage of pretraining and temporal modeling is necessary as each of them improved generalization on unseen participants.

In summary, this work highlights the advantage of combining temporal and modality-aware modeling with multiple pretraining strategies for enhancing the transferability of multimodal representations in biosignal-based driver drowsiness detection. The proposed architecture and training pipeline offer a scalable foundation for further research in fatigue monitoring and robust physiological signal representation learning. Moreover, the findings of this work pose a new standard for the ongoing internal engineering efforts of Robert Bosch Ltd./Bosch Group, Hungary to advance the development of adaptive driver assistance systems and other intelligent interfaces reliant on the interpretation of continuous biosignals.

# Acknowledgements

I would like to thank the Robert Bosch Ltd./Bosch Group, Hungary, for their professional support of the research that this study is based on.

# Appendix A

## Configuration Parameters

This appendix lists the configurable parameters used in model training, along with their default values and brief descriptions.

### General Training Parameters

| Parameter            | Default Value | Description                                                                  |
|----------------------|---------------|------------------------------------------------------------------------------|
| seed                 | 42            | Random seed for reproducibility.                                             |
| learning_rate        | 1e-3          | Learning rate for optimizer (Adam).                                          |
| weight_decay         | 1e-5          | Weight decay (L2 regularization) parameter for optimizer.                    |
| batch_size           | 16            | Number of training samples per batch.                                        |
| effective_batch_size | 32            | Effective batch size considering gradient accumulation.                      |
| num_epochs           | 100           | Total number of training epochs.                                             |
| checkpoint_interval  | 1             | Interval (in epochs) at which model checkpoints are saved.                   |
| patience             | 10            | Early stopping patience: epochs to wait before stopping without improvement. |

For all training stages, the path to the training and validation data, index files, and weights of the pretrained architecture elements (if applicable) can be set in the configuration file as well.

## Unimodal Encoder Parameters

Example (EEG encoder shown; similar applies to ECG/RSP):

| Parameter           | Default Value  | Description                                                                                  |
|---------------------|----------------|----------------------------------------------------------------------------------------------|
| ch_num              | 8              | Number of EEG channels.                                                                      |
| embedding_dim       | 128            | Dimension of embedding vectors produced by the encoder.                                      |
| seq_len             | 15360 (256×60) | Input sequence length.                                                                       |
| pooling_size        | 8              | Pooling size for input embeddings.                                                           |
| downsample_factor   | 2              | Downsampling factor in convolutional layers.                                                 |
| input_kernel_size   | 7              | Kernel size for the convolutional layers.                                                    |
| conv_only           | False          | Whether the encoder uses convolutional layers only, excluding transformer layers.            |
| enc_num_heads       | 4              | Number of attention heads in context encoder.                                                |
| enc_layers_num      | 4              | Number of transformer layers in context encoder.                                             |
| enc_dim_ff          | 4*embed_dim    | Size of the intermediate feedforward layer in each Transformer block in the context encoder. |
| enc_dropout         | 0.1            | Dropout rate in context encoder.                                                             |
| patch_mask_ratio    | 0.5            | Ratio of masked patches during training.                                                     |
| visible_block_count | 10             | Number of visible blocks preserved during Semantic Subsequence Preserving.                   |
| pred_num_heads      | 4              | Number of attention heads in predictor.                                                      |
| pred_layers_num     | 4              | Number of transformer layers in predictor.                                                   |
| pred_dim_ff         | 4*embed_dim    | Size of the intermediate feedforward layer in each Transformer block in the predictor.       |

## Temporal Masked Modeling Parameters

|                  |             |                                                                                            |
|------------------|-------------|--------------------------------------------------------------------------------------------|
| pretrained       | True        | Whether pretrained modality encoders are used.                                             |
| frozen_epoch_num | 10          | Number of epochs while the pretrained unimodal encoder's weights are frozen.               |
| conv_only        | False       | Whether the modality encoder uses convolutional layers only, excluding transformer layers. |
| tmm_mask_ratio   | 0.5         | Ratio of segments to be masked out and predicted.                                          |
| tmm_mask_num     | 4           | Number of masked versions to be generated for each sample.                                 |
| tmm_window_len   | 16          | Number of 1-minute segment embeddings belonging to a single continuous input window.       |
| tmm_num_heads    | 4           | Number of attention heads in TMM encoder.                                                  |
| tmm_layers_num   | 1           | Number of transformer layers in TMM encoder.                                               |
| tmm_dim_ff       | 2*embed_dim | Size of the intermediate feedforward layer in each Transformer block in the TMM encoder.   |
| tmm_dropout      | 0.1         | Dropout rate in TMM encoder.                                                               |
| mask_loss_coeff  | 1           | Coefficient of masking loss in overall loss.                                               |
| cov_loss_coeff   | 1           | Coefficient of covariance loss in overall loss.                                            |
| std_loss_coeff   | 1           | Coefficient of standard deviation loss in overall loss.                                    |

## Fusion Parameters

|                       |                      |                                                                               |
|-----------------------|----------------------|-------------------------------------------------------------------------------|
| pretrained            | True                 | Whether pretrained unimodal encoders are used.                                |
| conv_only             | False                | Whether the unimodal encoders use convolutional layers only.                  |
| TMM_used              | True                 | Whether the modality encoders were fine-tuned with the TMM task.              |
| fusion_mode           | "FFN"                | Method of multimodal fusion ("FFN" or "Transformer").                         |
| num_modalities        | 3                    | Number of modalities to fuse.                                                 |
| contrastive_loss_type | "temporal"           | Type of contrastive loss ("exact_match" or "temporal").                       |
| positive_radius       | 10                   | Maximum temporal distance (in minutes) to consider samples positive pairs.    |
| sim_loss_coeff        | 10                   | Coefficient of similarity loss in overall loss.                               |
| cov_coeff             | 100                  | Coefficient of covariance loss in overall loss.                               |
| std_coeff             | 10                   | Coefficient of standard deviation loss in overall loss.                       |
| fusion_num_heads      | 4                    | Number of attention heads in the fusion transformer encoder.                  |
| fusion_layers_num     | 1                    | Number of transformer layers in the fusion transformer encoder.               |
| fusion_dim_ff         | 2*embed_dim          | Size of the intermediate feedforward layer in the fusion transformer encoder. |
| fusion_dropout        | 0.1                  | Dropout rate in the fusion transformer encoder.                               |
| dimensions            | [384, 256, 256, 128] | Dimensions of the FNN's layers.                                               |

## Classification Parameters

|                   |                         |                                                                             |
|-------------------|-------------------------|-----------------------------------------------------------------------------|
| pretrained        | True                    | Whether pretrained modality encoders are used.                              |
| conv_only         | False                   | Whether the modality encoders use convolutional layers only.                |
| TMM_used          | True                    | Whether the modality encoders were fine-tuned with the TMM task.            |
| fusion_pretrained | True                    | Whether pretrained fusion encoder is used.                                  |
| combination_mode  | "diff"                  | Method of combining two embeddings for classification ("diff" or "concat"). |
| cls_hidden_dim    | $\text{embed\_dim} / 2$ | Size of the hidden feedforward layer in the classifier head.                |
| num_classes       | 2                       | Number of classes to predict.                                               |
| dropout           | 0.4                     | Dropout rate in the classifier head.                                        |



# Bibliography

- [1] World Health Organization. *Road Traffic Injuries*. Accessed: 2025-02-18. World Health Organization. 2023. URL: <https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries>.
- [2] Brian C Tefft. *Asleep at the wheel: The prevalence and impact of drowsy driving*. Accessed: 2025-04-02. 2010. URL: <https://trid.trb.org/View/987584>.
- [3] Mohamed Hedi Baccour et al. “Camera-Based Eye Blink Detection Algorithm for Assessing Driver Drowsiness”. In: *2019 IEEE Intelligent Vehicles Symposium (IV)*. 2019, pp. 987–993. DOI: 10.1109/IVS.2019.8813871.
- [4] Sumeet Saurav et al. “Yawn Detection for Driver’s Drowsiness Prediction Using Bi-Directional LSTM with CNN Features”. In: *Intelligent Human Computer Interaction*. Ed. by Uma Shanker Tiwary and Santanu Chaudhury. Springer International Publishing, 2020, pp. 189–200. ISBN: 978-3-030-44689-5.
- [5] Gao Zhenhai et al. “Driver Drowsiness Detection Based on Time Series Analysis of Steering Wheel Angular Velocity”. In: *2017 9th International Conference on Measuring Technology and Mechatronics Automation (ICMTMA)*. 2017, pp. 99–101. DOI: 10.1109/ICMTMA.2017.0031.
- [6] T. Pilutti and A.G. Ulsoy. “Identification of driver state for lane-keeping tasks”. In: *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans* 29.5 (1999), pp. 486–502. DOI: 10.1109/3468.784175.
- [7] Ke Lu et al. “Detecting driver fatigue using heart rate variability: A systematic review”. In: *Accident Analysis & Prevention* 178 (2022), p. 106830. ISSN: 0001-4575. DOI: <https://doi.org/10.1016/j.aap.2022.106830>. URL: <https://www.sciencedirect.com/science/article/pii/S0001457522002652>.
- [8] Hafeez Ur Rehman Siddiqui et al. “Non-Invasive Driver Drowsiness Detection System”. In: *Sensors* 21.14 (2021). ISSN: 1424-8220. DOI: 10.3390/s21144833.

- [9] Torbjörn Åkerstedt and Mats Gillberg. “Subjective and objective sleepiness in the active individual”. In: *International journal of neuroscience* 52.1-2 (1990), pp. 29–37.
- [10] Jongseong Gwak, Akinari Hirao, and Motoki Shino. “An Investigation of Early Detection of Driver Drowsiness Using Ensemble Machine Learning Based on Hybrid Sensing”. In: *Applied Sciences* 10.8 (2020). ISSN: 2076-3417. DOI: 10 . 3390/app10082890.
- [11] Prima Dewi Purnamasari et al. “Mobile EEG Based Drowsiness Detection using K-Nearest Neighbor”. In: *2019 IEEE 10th International Conference on Awareness Science and Technology (iCAST)*. 2019, pp. 1–5. DOI: 10 . 1109/ICAwST . 2019 . 8923161.
- [12] M. Murugappan et al. “Subtractive fuzzy classifier based driver drowsiness levels classification using EEG”. In: *2013 International Conference on Communication and Signal Processing*. 2013, pp. 159–164. DOI: 10 . 1109/iccsp . 2013 . 6577036.
- [13] Sadegh Arefnezhad et al. “Driver drowsiness estimation using EEG signals with a dynamical encoder–decoder modeling framework”. In: *Scientific Reports* 12.1 (2022), p. 2650. ISSN: 2045-2322. DOI: 10 . 1038/s41598-022-05810-x. URL: <https://doi.org/10.1038/s41598-022-05810-x>.
- [14] Y. Lecun et al. “Gradient-based learning applied to document recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324. DOI: 10 . 1109/5 . 726791.
- [15] Jeffrey L. Elman. “Finding Structure in Time”. In: *Cognitive Science* 14.2 (1990), pp. 179–211. DOI: [https : / / doi . org / 10 . 1207 / s15516709cog1402 \](https://doi.org/10.1207/s15516709cog1402_1) \_1. URL: [https : / / onlinelibrary . wiley . com / doi / abs / 10 . 1207 / s15516709cog1402\\_1](https://onlinelibrary.wiley.com/doi/abs/10.1207/s15516709cog1402_1).
- [16] Hong Zeng et al. “EEG classification of driver mental states by deep learning”. In: *Cognitive Neurodynamics* 12.6 (2018), pp. 597–606. ISSN: 1871-4099. DOI: 10 . 1007/s11571-018-9496-y. URL: <https://doi.org/10.1007/s11571-018-9496-y>.
- [17] Eric Juwei Cheng, Ku-Young Young, and Chin-Teng Lin. “Image-based EEG signal processing for driving fatigue prediction”. In: *2018 International Automatic Control Conference (CACS)*. 2018, pp. 1–5. DOI: 10 . 1109/CACS . 2018 . 8606734.

- [18] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [19] Guillaume Geoffroy et al. “Drowsiness Detection Using Joint EEG-ECG Data With Deep Learning”. In: *2021 29th European Signal Processing Conference (EUSIPCO)*. 2021, pp. 955–959. DOI: 10.23919/EUSIPCO54536.2021.9616046.
- [20] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: [https://proceedings.neurips.cc/paper\\_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf).
- [21] Zhijiang Wan et al. “EEGformer: A transformer-based brain activity classification method using EEG signal”. In: *Frontiers in Neuroscience* Volume 17 - 2023 (2023). ISSN: 1662-453X. DOI: 10.3389/fnins.2023.1148855. URL: <https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2023.1148855>.
- [22] Guang Shi, Zheng Chen, and Renyuan Zhang. “A Transformer-Based Spatial-Temporal Sleep Staging Model Through Raw EEG”. In: *2021 International Conference on High Performance Big Data and Intelligent Systems (HPBDIS)*. 2021, pp. 110–115. DOI: 10.1109/HPBDIS53214.2021.9658439.
- [23] Ayman Anwar et al. “Transformers in biosignal analysis: A review”. In: *Information Fusion* 114 (2025), p. 102697. ISSN: 1566-2535. DOI: <https://doi.org/10.1016/j.inffus.2024.102697>.
- [24] Yonghao Song et al. “EEG Conformer: Convolutional Transformer for EEG Decoding and Visualization”. In: *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 31 (2023), pp. 710–719. DOI: 10.1109/TNSRE.2022.3230250.
- [25] Zhenyuan Lu, Burcu Ozek, and Sagar Kamarthi. “Transformer encoder with multiscale deep learning for pain classification using physiological signals”. In: *Frontiers in Physiology* Volume 14 - 2023 (2023). ISSN: 1664-042X. DOI: 10.3389/fphys.2023.1294577.
- [26] Essam Debie et al. “Multimodal Fusion for Objective Assessment of Cognitive Workload: A Review”. In: *IEEE Transactions on Cybernetics* 51.3 (2021), pp. 1542–1555. DOI: 10.1109/TCYB.2019.2939399.

- [27] Yueying Zhou et al. “Cognitive Workload Recognition Using EEG Signals and Machine Learning: A Review”. In: *IEEE Transactions on Cognitive and Developmental Systems* 14.3 (2022), pp. 799–818. DOI: 10.1109/TCDS.2021.3090217.
- [28] Diego Candia-Rivera et al. “Cardiac sympathetic-vagal activity initiates a functional brain–body response to emotional arousal”. In: *Proceedings of the National Academy of Sciences* 119.21 (2022), e2119599119. DOI: 10.1073/pnas.2119599119.
- [29] Daniel S. Kluger, Joachim Gross, and Christian Keitel. “A Dynamic Link between Respiration and Arousal”. In: *Journal of Neuroscience* 44.47 (2024). ISSN: 0270-6474. DOI: 10.1523/JNEUROSCI.1173-24.2024.
- [30] Gyanendra K. Verma and Uma Shanker Tiwary. “Multimodal fusion framework: A multiresolution approach for emotion classification and recognition from physiological signals”. In: *NeuroImage* 102 (2014). Multimodal Data Fusion, pp. 162–172. ISSN: 1053-8119. DOI: <https://doi.org/10.1016/j.neuroimage.2013.11.007>.
- [31] Ruiqi Wang et al. “Husformer: A Multimodal Transformer for Multimodal Human State Recognition”. In: *IEEE Transactions on Cognitive and Developmental Systems* 16.4 (2024), pp. 1374–1390. DOI: 10.1109/TCDS.2024.3357618.
- [32] Serajeddin Ebrahimian et al. “Multi-Level Classification of Driver Drowsiness by Simultaneous Analysis of ECG and Respiration Signals Using Deep Neural Networks”. In: *International Journal of Environmental Research and Public Health* 19.17 (2022), p. 10736. ISSN: 1660-4601. DOI: 10.3390/ijerph191710736.
- [33] Chao Li et al. “Exploring temporal representations by leveraging attention-based bidirectional LSTM-RNNs for multi-modal emotion recognition”. In: *Information Processing Management* 57.3 (2020), p. 102185. ISSN: 0306-4573. DOI: <https://doi.org/10.1016/j.ipm.2019.102185>.
- [34] Ting Chen et al. “A Simple Framework for Contrastive Learning of Visual Representations”. In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. PMLR, 2020, pp. 1597–1607. URL: <https://proceedings.mlr.press/v119/chen20j.html>.

- [35] Kaiming He et al. “Momentum contrast for unsupervised visual representation learning”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2020, pp. 9729–9738.
- [36] Jean-Bastien Grill et al. “Bootstrap your own latent: A new approach to self-supervised learning”. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates Inc., 2020, pp. 21271–21284. ISBN: 9781713829546.
- [37] Hubert Banville et al. “Uncovering the structure of clinical EEG signals with self-supervised learning”. In: *Journal of Neural Engineering* 18.4 (2021), p. 046020. DOI: 10.1088/1741-2552/abca18. URL: <https://dx.doi.org/10.1088/1741-2552/abca18>.
- [38] Chaoqi Yang, M Brandon Westover, and Jimeng Sun. “BIOT: Biosignal Transformer for Cross-data Learning in the Wild”. In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: <https://openreview.net/forum?id=c2LZyTyddi>.
- [39] Shohreh Deldari et al. “CroSSL: Cross-modal Self-Supervised Learning for Time-series through Latent Masking”. In: *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. WSDM '24. Association for Computing Machinery, 2024, pp. 152–160. ISBN: 9798400703713. DOI: 10.1145/3616855.3635795.
- [40] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423.
- [41] Kaiming He et al. “Masked Autoencoders Are Scalable Vision Learners”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 16000–16009.
- [42] Navid Mohammadi Foumani et al. “EEG2Rep: Enhancing Self-supervised EEG Representation Through Informative Masked Inputs”. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. KDD '24. Association for Computing Machinery, 2024, pp. 5544–5555. ISBN: 9798400704901. DOI: 10.1145/3637528.3671600.

- [43] Demetres Kostas, Stephane Aroca-Ouellette, and Frank Rudzicz. “BENDR: Using transformers and a contrastive self-supervised learning task to learn from massive amounts of EEG data”. In: *Frontiers in Human Neuroscience* 15 (2021), p. 653659. ISSN: 1662-5161. DOI: 10.3389/fnhum.2021.653659.
- [44] Hsiang-Yun Sherry Chien et al. *MAEEG: Masked Auto-encoder for EEG Representation Learning*. Accessed: 2025-04-02. 2022. arXiv: 2211.02625 [eess.SP]. URL: <https://arxiv.org/abs/2211.02625>.
- [45] Adrien Bardes, Jean Ponce, and Yann LeCun. *VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning*. Accessed: 2025-04-02. 2022. arXiv: 2105.04906 [cs.CV]. URL: <https://arxiv.org/abs/2105.04906>.
- [46] Alec Radford et al. *Learning Transferable Visual Models From Natural Language Supervision*. Accessed: 2025-04-02. 2021. arXiv: 2103.00020 [cs.CV]. URL: <https://arxiv.org/abs/2103.00020>.
- [47] Fredrik Carlsson et al. “Semantic Re-tuning with Contrastive Tension”. In: *International Conference on Learning Representations*. 2021. URL: [https://openreview.net/forum?id=0v\\_sMNau-PF](https://openreview.net/forum?id=0v_sMNau-PF).
- [48] Shuyang Cao and Lu Wang. *CLIFF: Contrastive Learning for Improving Faithfulness and Factuality in Abstractive Summarization*. Accessed: 2025-04-02. 2021. arXiv: 2109.09209 [cs.CL]. URL: <https://arxiv.org/abs/2109.09209>.
- [49] Emadeldeen Eldele et al. *Time-Series Representation Learning via Temporal and Contextual Contrasting*. Accessed: 2025-04-02. 2021. arXiv: 2106.14112 [cs.LG]. URL: <https://arxiv.org/abs/2106.14112>.
- [50] Meihong Zhang et al. “Multimodal Vigilance Estimation With Modality-Pairwise Contrastive Loss”. In: *IEEE Transactions on Biomedical Engineering* 71.4 (2024), pp. 1139–1150. DOI: 10.1109/TBME.2023.3328942.
- [51] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. *Representation Learning with Contrastive Predictive Coding*. Accessed: 2025-04-02. 2019. arXiv: 1807.03748. URL: <https://arxiv.org/abs/1807.03748>.

- [52] Shohreh Deldari et al. “Time Series Change Point Detection with Self-Supervised Contrastive Predictive Coding”. In: *Proceedings of the Web Conference 2021*. WWW '21. Association for Computing Machinery, 2021, pp. 3124–3135. ISBN: 9781450383127. DOI: 10.1145/3442381.3449903. URL: <https://doi.org/10.1145/3442381.3449903>.
- [53] Shohreh Deldari et al. “COCOA: Cross Modality Contrastive Learning for Sensor Data”. In: *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 6.3 (2022). DOI: 10.1145/3550316. URL: <https://doi.org/10.1145/3550316>.
- [54] Shutoko Revival Project Team. *Shutoko Revival Project*. Accessed: 2025-04-02. 2023. URL: <https://shutokorevivalproject.com/>.
- [55] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.
- [56] Dominique Makowski et al. “NeuroKit2: A Python toolbox for neurophysiological signal processing”. In: *Behavior Research Methods* 53.4 (2021), pp. 1689–1696. DOI: 10.3758/s13428-020-01516-y. URL: <https://doi.org/10.3758/s13428-020-01516-y>.
- [57] Quentin Lhoest et al. “Datasets: A Community Library for Natural Language Processing”. In: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 2021, pp. 175–184. URL: <https://aclanthology.org/2021.emnlp-demo.21>.
- [58] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Accessed: 2025-04-02. 2019. arXiv: 1912.01703 [cs.LG]. URL: <https://arxiv.org/abs/1912.01703>.

# List of Figures

|     |                                                                            |    |
|-----|----------------------------------------------------------------------------|----|
| 2.1 | EEG2Rep architecture figure taken from the original paper[42]              | 16 |
| 3.1 | DrowsiRank architecture                                                    | 24 |
| 3.2 | Process of MoCA-T Pretraining                                              | 34 |
| 3.3 | Overview of the dataset creation pipeline                                  | 42 |
| 5.1 | Flowchart of chained pretraining stages for each model variant             | 64 |
| 5.2 | Relationship between Unseen Participant and Seen Participant Test Accuracy | 68 |

# List of Tables

|     |                                 |    |
|-----|---------------------------------|----|
| 5.1 | Classification results per fold | 66 |
| 5.2 | Averaged classification results | 67 |



# List of Algorithms

|   |                                                             |    |
|---|-------------------------------------------------------------|----|
| 1 | Forward pass of the Unimodal Encoder . . . . .              | 26 |
| 2 | Forward pass of the Unimodal Encoder during PaReM . . . . . | 30 |
| 3 | Loss Computation for Patch-Level Reconstruction . . . . .   | 30 |
| 4 | TMM-SegRep Pretraining Process . . . . .                    | 33 |
| 5 | Temporal Contrastive Loss . . . . .                         | 36 |
| 6 | MoCA-T Pretraining . . . . .                                | 37 |
| 7 | Forward pass of the DrowsiRank classifier . . . . .         | 38 |